

ORIX Education Series

ORIX 教育版四足机器狗 Python SDK 开发手册

面向教育场景的专业级机器人开发平台

大算机器人

Brainstem SDK for Python

版本 v1.0.0

日期 2026 年 4 月

文档信息

项目	说明
文档名称	ORIX 教育版四足机器狗 Python SDK 开发手册
版本号	v1.0.0
编写日期	2026 年 4 月
适用产品	ORIX 教育版四足机器狗
适用 SDK	brainstem_sdk (Python)

版本历史

版本	日期	作者	说明
v1.0.0	2026-04	大算机器人	正式发布

本文档为大算机器人内部技术文档，未经许可不得外传。

目录

前言	11
第一章 概述	12
1.1 产品定位	12
1.2 SDK 能做什么	12
1.3 典型开发流程	12
1.4 手册结构	13
第二章 快速开始	14
2.1 环境要求	14
2.2 安装	14
2.2.1 方式一：本地开发安装（推荐）	14
2.2.2 方式二：直接安装依赖	15
2.3 三行代码控制机器人	15
2.4 首次连接检查清单	15
2.5 网络配置	16
2.6 验证连接脚本	16
2.7 第一次完整操作	18
2.8 使用 with 语句	19
2.9 常见问题	19
2.9.1 连接超时	19
2.9.2 walk() 没反应	19
2.9.3 电机不动	19
2.9.4 操作被拒绝	20
第三章 理论基础	21
3.1 PD 位置控制	21
3.1.1 物理意义	21
3.1.2 稳定性条件	21
3.1.3 阻尼比分析	22
3.2 强化学习运动控制	22

3.2.1	策略网络	22
3.2.2	观测空间	22
3.2.3	历史帧堆叠	23
3.2.4	动作空间与缩放	23
3.2.5	Sim-to-Real	23
3.3	姿态表示与坐标变换	23
3.3.1	四元数基础	24
3.3.2	四元数到旋转矩阵	24
3.3.3	投影重力计算	24
3.3.4	Hamilton 约定与 ROS 约定	24
3.4	有限状态机	24
3.4.1	形式化定义	24
3.4.2	状态转移表	25
3.4.3	安全不变量	25
3.5	控制频率与实时性	25
3.5.1	单周期流水线	26
3.5.2	时间预算	26
3.5.3	为什么单线程事件循环足够	26
第四章	坐标系与关节	27
4.1	机体坐标系	27
4.1.1	正面视图	27
4.1.2	侧面视图（右侧）	27
4.1.3	walk() 指令与坐标系的关系	28
4.2	关节编号总表	28
4.3	关节分组	28
4.3.1	按腿分组	28
4.3.2	按关节类型分组	29
4.3.3	俯瞰图 (Top View)	29
4.3.4	侧面图——单腿结构	29
4.4	典型姿态	29
4.4.1	站立姿态 (Standing Pose)	29
4.4.2	坐下姿态 (Sitting Pose)	30
4.4.3	姿态可视化	30
4.5	PD 控制参数	30
4.5.1	推理阶段（行走时）	30
4.5.2	站立姿态	31
4.5.3	PD 控制接口	31

4.6	编程中的关节操作	32
4.6.1	读取关节数据	32
4.6.2	诊断单条腿	32
4.6.3	清除单腿故障	32
第五章	状态机与运动控制	33
5.1	状态机总览	33
5.1.1	状态图	33
5.1.2	五个状态	33
5.2	状态转换详解	34
5.2.1	Zero → Grounded	34
5.2.2	Grounded → Standing (站起)	34
5.2.3	Standing → Walking (开始行走)	34
5.2.4	Walking 中更新方向	35
5.2.5	Walking → Standing (停步)	35
5.2.6	Standing → Grounded (坐下)	35
5.2.7	Walking → Grounded (行走中直接坐下)	35
5.3	非法操作处理	36
5.4	walk() 速度约定	36
5.4.1	参数说明	36
5.4.2	常见用法	37
5.4.3	运动方向参考图	37
5.5	电机使能/禁用生命周期	37
5.5.1	完整生命周期	37
5.5.2	关键规则	38
5.5.3	安全的关机流程	38
5.6	控制权仲裁 (ControlArbiter)	38
5.6.1	优先级机制	38
5.6.2	仲裁规则	39
5.6.3	超时参数	39
5.6.4	SDK 开发建议	39
5.7	故障处理	39
5.7.1	Fault 状态转换	39
5.7.2	电机故障清除	39
5.7.3	低压保护	40
5.8	策略切换	40
5.8.1	查询和切换	40
5.8.2	切换规则	40

5.9	典型使用流程	40
5.9.1	标准操作序列	41
5.9.2	实时数据监控	42
5.9.3	安全包装器	43
5.10	关键参数速查	43
第六章	实用开发指南	45
6.1	电池管理	45
6.1.1	电池电量估算	45
6.1.2	周期性电池监控	45
6.2	数据记录与采集	46
6.2.1	记录 IMU 数据到 CSV	46
6.2.2	同时记录 IMU 和关节数据	47
6.3	远程部署	48
6.3.1	SSH 部署 workflow	48
6.4	集成计算机视觉	48
6.5	自定义 Profile	49
6.5.1	Profile 文件结构	49
6.5.2	部署自定义 Profile	50
6.6	开发建议与最佳实践	50
第七章	传感器数据	52
7.1	传感器总览	52
7.2	IMU 惯性测量单元	52
7.2.1	硬件与数据	52
7.2.2	坐标系	53
7.2.3	四元数约定	53
7.2.4	订阅 IMU 数据	53
7.2.5	数据解读	54
7.3	关节编码器	55
7.3.1	关节布局	55
7.3.2	数据内容	55
7.3.3	订阅关节数据	55
7.3.4	关节位置安全限位	56
7.4	电机状态	56
7.4.1	查询电机状态	56
7.4.2	MotorInfo 字段说明	57
7.4.3	故障码解读	57
7.4.4	清除电机故障	57

7.5	电压监控	57
7.5.1	读取电压	58
7.5.2	低压保护机制	58
7.5.3	电压监控最佳实践	58
7.6	综合示例：传感器数据记录到 CSV	59
第八章	API 参考	63
8.1	OrixClient 构造与连接	63
8.1.1	构造函数	63
8.1.2	close()	64
8.1.3	上下文管理器	64
8.2	电机控制	64
8.2.1	enable()	64
8.2.2	disable()	64
8.3	运动指令	65
8.3.1	walk()	65
8.3.2	stand_up()	66
8.3.3	sit_down()	66
8.4	状态查询	66
8.4.1	get_state()	66
8.4.2	get_voltage()	67
8.4.3	get_motor_status()	67
8.5	策略管理	68
8.5.1	get_profile()	68
8.5.2	switch_profile()	68
8.6	诊断与标定	69
8.6.1	clear_motor_fault()	69
8.6.2	set_zero()	69
8.7	实时数据流	70
8.7.1	listen_imu()	70
8.7.2	listen_joint()	70
8.7.3	listen_state()	70
8.8	数据类型参考	71
8.8.1	Vec3	71
8.8.2	Quat	71
8.8.3	ImuData	71
8.8.4	JointData	71
8.8.5	MotorInfo	72

8.8.6	ProfileInfo	72
8.8.7	RobotState	72
8.9	错误处理	73
8.9.1	连接超时	73
8.9.2	常见错误码	73
8.9.3	Arbiter 控制权冲突	74
8.9.4	流式接口异常处理	74
第九章	安全指南	77
9.1	操作前检查清单	77
9.2	低压保护	79
9.2.1	保护机制	79
9.2.2	电压等级参考	79
9.2.3	注意事项	79
9.3	关节限位保护	79
9.3.1	保护机制	79
9.3.2	触发后果	80
9.3.3	预防措施	80
9.4	电机故障处理	80
9.4.1	检测故障	80
9.4.2	清除故障	80
9.4.3	故障处理流程	80
9.4.4	反复出现的故障	81
9.5	急停程序	81
9.5.1	软件急停 (SDK)	81
9.5.2	遥控器急停	81
9.5.3	硬件断电	81
9.5.4	急停优先级	82
9.6	Arbiter 控制仲裁安全	82
9.6.1	仲裁规则	82
9.6.2	对 SDK 开发者的影响	82
9.6.3	安全启示	82
9.7	禁止操作	83
9.8	标准关机序列	83
9.9	故障诊断树	84
第十章	常见问题	86
10.1	连接问题	86
10.1.1	Q: 连接超时 (DEADLINE_EXCEEDED)	86

10.1.2 Q: 连接被拒绝 (UNAVAILABLE)	87
10.1.3 Q: IP 地址不确定	87
10.2 电机不动	88
10.2.1 Q: 调用了 enable() 但电机没反应	88
10.2.2 Q: 只有部分电机动, 其他不动	88
10.2.3 Q: enable() 后状态没变化	89
10.3 机器人摔倒	89
10.3.1 Q: 行走中突然坐下	89
10.3.2 Q: 站起来后立即摔倒	90
10.4 行走指令无效	90
10.4.1 Q: walk() 调用成功但机器人不动	90
10.4.2 Q: 遥控器在线时 SDK 指令被忽略	90
10.5 策略切换失败	91
10.5.1 Q: switch_profile() 报错	91
10.5.2 Q: 策略名称不存在	91
10.5.3 Q: 切换策略后行为异常	92
10.6 IMU 数据异常	92
10.6.1 Q: 四元数数值看起来不对	92
10.6.2 Q: 机器人静止但角速度不为零	92
10.6.3 Q: listen_imu() 频率不是 50Hz	93
10.7 性能与延迟	93
10.7.1 Q: gRPC 调用延迟有多大?	93
10.7.2 Q: 50Hz 控制频率够用吗?	93
10.7.3 Q: Python GIL 会影响实时性吗?	94
10.8 环境变量参考	94
10.9 网络排障	95
10.9.1 基本连通性检查	95
10.9.2 防火墙检查	95
10.9.3 网络拓扑要求	96
10.9.4 远程开发 (通过 SSH 隧道)	96
10.10 提交 Bug 报告	96
10.10.1 必要信息	96
10.10.2 收集诊断信息的脚本	97
10.10.3 日志收集	99
10.10.4 联系方式	99
10.11 SSH 访问与系统服务	99
10.11.1 默认账户	99
10.11.2 brainstem 服务管理	100

10.11.3 无网络环境下的备选连接方式	100
---------------------------------	-----

前言

近年来，国家高度重视人工智能与机器人教育的发展。《新一代人工智能发展规划》明确要求在中小学和高等院校开设人工智能相关课程，《教育部关于加强和改进中小学实验教学的意见》强调动手实践和工程思维的培养，教育部《人工智能教育基地建设指南》进一步推动了 AI+ 机器人实验室在全国高校和中学的普及。在这一大背景下，实物机器人作为连接理论与实践的桥梁，正在成为教育体系的重要组成部分。

四足机器人是一类典型的高集成度教学平台。它融合了机械工程（腿部机构设计、运动学建模）、电子控制（伺服驱动、传感器融合）、嵌入式系统（实时控制、CAN 总线通信）、计算机视觉（姿态估计、环境感知）和人工智能（强化学习策略训练、sim-to-real 迁移）等多个学科的核心内容。相较于传统的机械臂或移动底盘，四足机器人的动力学更加丰富，控制问题更具挑战性，能够让学生直观感受到理论与工程之间的差距，也能作为强化学习、仿生运动控制、自主导航等前沿技术的实验载体。

ORIX 教育版四足机器狗是大机器人面向教育场景推出的专业级开发平台。我们的设计遵循四个核心理念：**学以致用**——它不是展会上的观摩样品，而是一台可以编程、调参、训练自定义策略的真实机器人；**降低门槛**——3 行 Python 代码即可完成从连接到行走的全流程，学生不需要先掌握通信协议和控制理论；**不设天花板**——从入门的遥控行走到高级的 RL 策略训练、多传感器融合、SLAM 建图和自主导航，一个平台覆盖从本科到研究生的完整学习路径；**工业级可靠**——采用与工业机器人相同的 FSM 状态机、控制权仲裁和 E-Stop 安全机制，学生接触的是真实的工程实践。

本手册是 ORIX Python SDK 的完整开发指南，面向四类读者：高校教师（了解平台能力、设计课程实验方案）、学生开发者（从零开始学习机器人编程、完成课程项目和毕业设计）、科研人员（利用 SDK 传感器接口和控制 API 进行算法验证）以及竞赛团队（快速搭建原型参加 RoboCup、全国大学生机器人大赛等赛事）。手册覆盖从环境安装到高级开发的完整流程，建议首次使用时按顺序阅读前三章（前言、概述、快速开始），完成首次连通后再根据兴趣阅读理论基础和 API 参考。

大机器人技术团队 2026 年 4 月

Chapter 1

概述

本章导读

本章建立对 ORIX 平台的整体认识：它解决什么问题、能做什么、适合什么场景、后续章节如何组织。阅读后您将对整个 SDK 有全局认识，便于根据自己的目标决定详读哪些章节。

阅读建议：如果您想尽快跑通机器人控制，可跳到第 二 章；如果您关心控制理论，可直接读第 三 章。

1.1 产品定位

ORIX 教育版四足机器人是一个集成了运动控制、传感器融合、强化学习策略推理和安全机制的完整机器人平台。它由机器人本体（16 DOF 四足机器人）、Python SDK (`brainstem-sdk`) 和开发文档三部分组成。机器人内置完整的控制系统，开发者通过 SDK 在开发电脑上编写 Python 代码远程控制机器人，无需接触底层 C++ 或嵌入式代码。

1.2 SDK 能做什么

`brainstem-sdk` 提供了对机器人的全方位控制能力，主要包括以下五类接口：

- 运动控制 — 使能/禁用电机、站立、坐下、行走、切换速度模式、播放预设动作
- 传感器读取 — IMU 姿态数据流（50Hz）、关节编码器数据流、电机状态、母线电压
- 状态管理 — 查询 FSM 状态、订阅状态变化事件、切换运动策略
- 诊断与维护 — 读取电机故障码、清除故障、电机标零、健康检查
- 摄像头接口 — USB 摄像头拍照、录像、MJPEG 流媒体服务（可选模块）

整个 SDK 遵循简洁设计原则：客户代码完全不需要了解 gRPC、protobuf、状态机实现细节，所有复杂性隐藏在 `OrixClient` 类后面。

1.3 典型开发流程

一个典型的 ORIX 开发流程包含以下步骤：

1. 环境准备 — 开发电脑装 Python 3.8+, 机器狗开机并连接局域网（详见第 二 章）
2. 安装 SDK — `pip install brainstem-sdk`
3. 验证连接 — 运行验证脚本，确认与机器狗通信正常
4. 编写控制代码 — 使用 `OrixClient` 发送运动指令和读取传感器数据
5. 调试与迭代 — 根据需要切换策略、调整参数、查看日志
6. 高级开发 — 集成计算机视觉、训练自定义 RL 策略、与 ROS2 对接

1.4 手册结构

本手册共 9 章，按” 入门 → 理论 → 参考 → 运维” 的顺序组织：

章	主题	适合谁
1	概述（本章）	所有读者
2	快速开始	第一次使用的开发者
3	理论基础	希望理解控制原理的学生/研究人员
4	硬件概览	需要了解硬件参数的工程师
5	坐标系与关节	进行运动规划的开发者
6	状态机与运动控制	编写控制逻辑的开发者
7	传感器数据	进行数据采集和算法研究的开发者
8	API 参考	查阅接口细节时的所有开发者
9	安全指南	上手前必读
10	常见问题	遇到问题时查阅

首次使用建议按顺序阅读第 1-2 章（概述 + 快速开始），完成首次连通和基本控制验证。之后可根据需要跳读：如果关心控制理论，读第 3 章；如果要做运动规划，读第 5-6 章；如果要开发传感器融合或 RL 策略，读第 7 章；遇到具体接口问题查第 8 章。安全指南（第 9 章）在真机操作前必读。

Chapter 2

快速开始

本文档帮助您在 5 分钟内完成 ORIX SDK 的安装、连接和第一次控制。

本章导读

本章以“先跑通、再理解”为目标，带您完成安装、网络连通性验证和第一次完整动作闭环。即使您已经有机机器人经验，也建议至少执行一次连接检查脚本。

阅读建议：按“环境要求 -> 安装 -> 首次连接检查清单 -> 验证连接脚本 -> 第一次完整操作”的顺序阅读。不要跳过安全提醒和状态机说明。

2.1 环境要求

在开始安装 SDK 之前，请确认您的开发环境满足以下最低要求。ORIX SDK 基于纯 gRPC 通信，不依赖 ROS 或其他中间件。

项目	要求
Python	≥ 3.8
操作系统	Windows / Linux / macOS
网络	开发机与机器人在同一局域网
端口	TCP 13145 (gRPC over HTTP/2)

注意：不需要安装 ROS、消息中间件或其他额外组件。SDK 通过纯 gRPC 直连机器人。

2.2 安装

2.2.1 方式一：本地开发安装（推荐）

推荐使用可编辑模式安装，便于在开发过程中随时修改 SDK 源码并立即生效。

```
1 cd sdk/python
2 pip install -e .
```

2.2.2 方式二：直接安装依赖

如果只需使用 SDK 而不修改源码，可以直接安装核心依赖包。

```
1 pip install grpcio>=1.50.0 protobuf>=4.21.0
```

安装完成后验证：

```
1 python -c "from brainstem_sdk import OrixClient; print('OK')"
```

输出 OK 即安装成功。

2.3 三行代码控制机器人

下面这段代码适合作为“环境和链路是否通了”的最小验证，而不是正式的运动脚本。正式任务中，您仍然需要补齐使能、电机状态确认、站起和安全收尾等步骤。

```
1 from brainstem_sdk import OrixClient
2
3 dog = OrixClient("192.168.66.190")
4 dog.walk(vx=0.3) # 30% 速度前进
```

这三行代码做了以下事情：

1. 导入 SDK
2. 通过 gRPC 连接到机器人（默认端口 13145）
3. 发送前进指令

注意：实际使用中，您需要先 `enable()` 使能电机并 `stand_up()` 站起来，才能执行 `walk()`。完整的操作流程见下方“第一次完整操作”。

2.4 首次连接检查清单

在第一次连接机器人之前，请逐项确认：

- ☐ 机器人已上电，电池电压正常（正常工作电压约 24V，低压保护 18V）
- ☐ 机器人主控板（S100P）已启动完成（约 30 秒）
- ☐ brainstem 服务已运行（`systemctl status brainstem`）
- ☐ 开发机与机器人在同一局域网（可以 `ping 192.168.66.190`）
- ☐ 遥控器已关闭或置于空闲状态（遥控器优先级高于 SDK）
- ☐ 机器人放置在平坦地面上，四足触地

2.5 网络配置

开发机（PC 或笔记本，IP 为 192.168.66.xxx）与 ORIX 机器人（IP 为 192.168.66.190）需处于同一局域网内，通过以太网或 WiFi 连接。SDK 使用 TCP 端口 13145 进行 gRPC 通信，无需额外中间件。

参数	默认值	说明
机器人 IP	192.168.66.190	S100P 主控板默认地址
gRPC 端口	13145	无需修改
协议	gRPC (HTTP/2)	无加密（局域网内使用）
超时	5.0 秒	可通过 <code>timeout</code> 参数调整

如果机器人 IP 不同，连接时指定即可：

```
1 dog = OrixClient("10.0.0.100", port=13145, timeout=5.0)
```

2.6 验证连接脚本

首次连接机器狗时，建议使用以下验证脚本逐项检查通信状态。该脚本会自动查询状态机、电压、电机和策略信息，帮助您快速确认连接是否正常。它的价值不只是“能不能连上”，还在于能快速判断问题属于网络、服务、控制权还是硬件在线状态。将以下脚本保存为 `verify_connection.py` 并运行：

```
1 """验证与 ORIX 机器人的连接。"""
2
3 from brainstem_sdk import OrixClient
4
5 ROBOT_IP = "192.168.66.190"
6
7 print(f"正在连接 ORIX @ {ROBOT_IP}:13145 ...")
8
9 try:
10     dog = OrixClient(ROBOT_IP, timeout=3.0)
11
12     # 1. 查询状态机
13     state = dog.get_state()
14     print(f"[OK] 状态机: {state}")
15
16     # 2. 查询电压
17     voltages = dog.get_voltage()
18     if voltages:
```



```

19     avg_v = sum(voltages) / len(voltages)
20     print(f"[OK] 平均电压: {avg_v:.1f}V ({len(voltages)} 个电机)")
21 else:
22     print("[OK] 电压查询成功 (无数据)")
23
24     # 3. 查询电机状态
25     motors = dog.get_motor_status()
26     online = sum(1 for m in motors if m.online)
27     print(f"[OK] 电机在线: {online}/{len(motors)}")
28
29     # 4. 查询当前策略
30     profile = dog.get_profile()
31     print(f"[OK] 当前策略: {profile.current}")
32     print(f"        可用策略: {profile.available}")
33
34     print("\n--- 连接验证通过 ---")
35
36     dog.close()
37
38 except Exception as e:
39     print(f"\n[FAIL] 连接失败: {e}")
40     print("\n排查步骤:")
41     print("  1. 确认机器人已上电并启动完成")
42     print("  2. 确认网络连通: ping 192.168.66.190")
43     print("  3. 确认服务运行: ssh sunrise@192.168.66.190 'systemctl\n        status brainstem'")
44     print("  4. 确认端口可达: 无防火墙阻断 13145/TCP")

```

运行:

```
1 python verify_connection.py
```

预期输出:

```

正在连接 ORIX @ 192.168.66.190:13145 ...
[OK] 状态机: grounded
[OK] 平均电压: 25.5V (16 个电机)
[OK] 电机在线: 16/16
[OK] 当前策略: default
        可用策略: ['default', 'mini']

--- 连接验证通过 ---

```

2.7 第一次完整操作

确认连接验证通过后，尝试完整的站起-行走-坐下流程。这个流程比“三行代码”更接近真实使用场景，因为它包含了状态切换等待、停步和安全收尾，是后续编写课堂演示脚本、实验脚本和自动化测试脚本的推荐模板。

```
1 import time
2 from brainstem_sdk import OrixClient
3
4 dog = OrixClient("192.168.66.190")
5
6 # 使能电机
7 dog.enable()
8 print(f"状态: {dog.get_state()}")    # grounded
9
10 # 站起来（约 3 秒过渡）
11 dog.stand_up()
12 time.sleep(4)
13 print(f"状态: {dog.get_state()}")    # standing
14
15 # 前进
16 dog.walk(vx=0.3)
17 time.sleep(3)
18
19 # 停步
20 dog.walk()    # 无参数 = 停步
21 time.sleep(1)
22
23 # 坐下（约 3 秒过渡）
24 dog.sit_down()
25 time.sleep(4)
26 print(f"状态: {dog.get_state()}")    # grounded
27
28 # 禁用电机
29 dog.disable()
30 dog.close()
```

注意：首次操作建议在台架上进行，或有人在旁边准备随时用遥控器接管。遥控器具有最高优先级，任何时候拨动遥控器都可以立即接管控制权。

2.8 使用 with 语句

为了避免忘记关闭连接，SDK 支持 Python 上下文管理器（with 语句）。退出 with 块时会自动释放 gRPC 通道资源，这是推荐的资源管理方式。

```
1 from brainstem_sdk import OrixClient
2
3 with OrixClient("192.168.66.190") as dog:
4     state = dog.get_state()
5     print(f"当前状态: {state}")
6 # 退出 with 块时自动调用 dog.close()
```

2.9 常见问题

2.9.1 连接超时

如果出现以下连接超时错误，通常是网络配置或服务未启动导致的。

```
grpc._channel._InactiveRpcError: ... UNAVAILABLE
```

排查步骤：

1. 确认机器人已上电且启动完成
2. ping 192.168.66.190 确认网络连通
3. SSH 登录确认服务运行: `ssh sunrise@192.168.66.190 'systemctl status brainstem'`
4. 检查防火墙是否阻断了 13145/TCP

2.9.2 walk() 没反应

行走指令未生效是新手最常遇到的问题之一。必须按照状态机顺序操作：`enable()` → `stand_up()` → 等待站稳 → `walk()`。

```
1 state = dog.get_state()
2 print(state) # 如果不是 "standing", walk() 不会生效
```

2.9.3 电机不动

电机无响应可能由多种原因引起，请按以下顺序逐一排查：

1. 确认已调用 `dog.enable()`
2. 遥控器在线时会抢占控制权——关闭或放下遥控器
3. 查看电机状态：`dog.get_motor_status()`

2.9.4 操作被拒绝

当状态机正在执行过渡动画时，新的运动指令会被拒绝。这是正常的安全保护行为。

```
grpc._channel._InactiveRpcError: ... transition in progress
```

状态机正在过渡中（如站起/坐下动画），请等待过渡完成后再发送新指令。

Chapter 3

理论基础

本章导读

本章介绍 ORIX 机器狗控制系统涉及的核心理论，包括 PD 控制、强化学习策略、姿态表示和状态机形式化定义。理解这些理论有助于更好地使用 SDK 进行二次开发和算法研究。

阅读建议：如果您只需要快速上手控制机器狗，可以跳过本章，直接阅读 API 参考章节。

3.1 PD 位置控制

四足机器人的每个关节电机均采用 PD（比例-微分）位置控制器。PD 控制器根据目标位置与当前位置的偏差，以及目标速度与当前速度的偏差，计算驱动关节所需的力矩。

其基本控制律为：

$$\tau = K_p (q_{\text{target}} - q_{\text{current}}) + K_d (\dot{q}_{\text{target}} - \dot{q}_{\text{current}}) \quad (3.1)$$

其中：

- τ —— 关节输出力矩（N·m）
- K_p —— 比例增益（位置刚度），单位 N·m/rad
- K_d —— 微分增益（速度阻尼），单位 N·m·s/rad
- $q_{\text{target}}, q_{\text{current}}$ —— 目标与当前关节角度（rad）
- $\dot{q}_{\text{target}}, \dot{q}_{\text{current}}$ —— 目标与当前关节角速度（rad/s）

3.1.1 物理意义

K_p 决定了关节的**刚度**： K_p 越大，关节对位置偏差的响应越强，如同弹簧越硬。 K_d 决定了关节的**阻尼**： K_d 越大，关节运动越平滑，振荡越少，如同液压阻尼器。

3.1.2 稳定性条件

PD 控制器稳定的必要条件为：

$$K_p > 0, \quad K_d > 0 \quad (3.2)$$

3.1.3 阻尼比分析

将单关节建模为二阶系统 $I\ddot{q} + K_d\dot{q} + K_pq = 0$ (I 为关节转动惯量), 可定义阻尼比 ζ 和固有频率 ω_n :

$$\omega_n = \sqrt{\frac{K_p}{I}} \quad (3.3)$$

$$\zeta = \frac{K_d}{2\sqrt{I \cdot K_p}} \quad (3.4)$$

不同阻尼比对应不同的动态行为:

- $\zeta < 1$ (欠阻尼): 关节到达目标位置时会出现振荡, 适用于需要快速响应的场景, 但可能引起机械抖动。
- $\zeta = 1$ (临界阻尼): 关节以最快速度到达目标位置且不超调。这是理论上最优的阻尼比。
- $\zeta > 1$ (过阻尼): 关节缓慢趋近目标位置, 无超调但响应速度慢。

在实际机器人中, 通常选取略欠阻尼 ($\zeta \approx 0.7 \sim 0.9$) 的参数, 以兼顾响应速度和稳定性。

3.2 强化学习运动控制

ORIX 采用基于强化学习 (Reinforcement Learning, RL) 的方法训练四足步态策略。策略网络在高保真物理仿真器 (Isaac Lab) 中通过数百万次交互学习, 然后直接部署到真实机器人上 (Sim-to-Real)。

3.2.1 策略网络

策略网络 π 是一个将观测映射到动作的神经网络:

$$\mathbf{a}_t = \pi(\mathbf{o}_t) \quad (3.5)$$

其中 $\mathbf{o}_t$ 为 t 时刻的观测向量, $\mathbf{a}_t$ 为输出动作。

3.2.2 观测空间

单帧观测向量维度为 57, 概念上由以下部分组成:

$$\mathbf{o} = \begin{bmatrix} \boldsymbol{\omega} \\ \mathbf{g}_{\text{proj}} \\ \mathbf{c}_{\text{cmd}} \\ \mathbf{q}_{\text{pos}} \\ \dot{\mathbf{q}} \\ \mathbf{a}_{\text{prev}} \\ \mathbf{b}_{\text{contact}} \end{bmatrix} \in \mathbb{R}^{57} \quad (3.6)$$

各分量含义如下：

分量	维度	说明
ω	3	陀螺仪角速度（机体坐标系）
$\mathbf{g}_{\text{proj}}$	3	投影重力向量（见 3.3 节）
$\mathbf{c}_{\text{cmd}}$	3	速度指令 (v_x, v_y, ω_z)
$\mathbf{q}_{\text{pos}}$	12	关节位置（12 腿关节，rad）
$\dot{\mathbf{q}}$	12	关节速度（rad/s）
$\mathbf{a}_{\text{prev}}$	12	上一时刻动作输出
$\mathbf{b}_{\text{contact}}$	12	足端接触布尔量

注：本章描述的是概念模型，实际部署版本的观测向量和动作维度可能因 Profile 配置和 ONNX 模型而异。具体维度请通过 `model_info` 接口查询或参考您训练时使用的 env config。

3.2.3 历史帧堆叠

为了让策略感知时序动态信息，通常将连续 H 帧观测堆叠为一个输入：

$$\mathbf{O}_t = \begin{bmatrix} \mathbf{o}_t & \mathbf{o}_{t-1} & \cdots & \mathbf{o}_{t-H+1} \end{bmatrix} \in \mathbb{R}^{57 \times H} \quad (3.7)$$

典型值 $H = 5$ ，因此策略网络的实际输入维度为 $57 \times 5 = 285$ 。

3.2.4 动作空间与缩放

策略网络输出 12 个腿关节的位置增量（另有 4 个脚轮关节由独立逻辑控制）。原始网络输出 $\mathbf{a} \in [-1, 1]^{12}$ 经缩放后变为目标关节角度：

$$q_{\text{target},i} = a_i \times \text{actionScale}_i + q_{\text{stand},i} \quad (3.8)$$

其中 `actionScale` 为每个关节的缩放系数，`qstand` 为默认站立姿态角度。

3.2.5 Sim-to-Real

策略在 Isaac Lab 仿真环境中训练，使用域随机化（Domain Randomization）缩小仿真与现实的差距。训练完成后，策略网络导出为 ONNX 格式，部署到真实机器人的嵌入式处理器上进行实时推理。

3.3 姿态表示与坐标变换

机器人的姿态（朝向）在三维空间中通常用四元数表示，它避免了欧拉角的万向锁问题。

3.3.1 四元数基础

单位四元数 $\mathbf{q}$ 定义为：

$$\mathbf{q} = (w, x, y, z), \quad \|\mathbf{q}\| = \sqrt{w^2 + x^2 + y^2 + z^2} = 1 \quad (3.9)$$

3.3.2 四元数到旋转矩阵

四元数 $\mathbf{q} = (w, x, y, z)$ 对应的旋转矩阵 R 为：

$$R = \begin{pmatrix} 1 - 2(y^2 + z^2) & 2(xy - wz) & 2(xz + wy) \\ 2(xy + wz) & 1 - 2(x^2 + z^2) & 2(yz - wx) \\ 2(xz - wy) & 2(yz + wx) & 1 - 2(x^2 + y^2) \end{pmatrix} \quad (3.10)$$

3.3.3 投影重力计算

投影重力（Projected Gravity）是将世界坐标系的重力方向变换到机体坐标系，用于让策略感知机器人当前的倾斜状态。其计算公式为：

$$\mathbf{g}_{\text{body}} = R^T \begin{pmatrix} 0 \\ 0 \\ -1 \end{pmatrix} \quad (3.11)$$

其中 R 为由 IMU 四元数计算得到的旋转矩阵。当机器人水平放置时， $\mathbf{g}_{\text{body}} = (0, 0, -1)^T$ ；当机器人倾斜时，重力在机体 x 、 y 轴上会产生分量。

3.3.4 Hamilton 约定与 ROS 约定

两种常见的四元数排列约定：

- **Hamilton 约定**（ORIX / Protobuf）： (w, x, y, z) ——实部 w 在前
- **ROS 约定**（ROS2 / tf2）： (x, y, z, w) ——实部 w 在后

在 ORIX 系统内部统一使用 Hamilton 约定。当与 ROS 系统交互时（如导航模块），`han_dog_bridge` 会自动完成两种约定之间的转换。

3.4 有限状态机

ORIX 的运动控制流程由一个有限状态机（Finite State Machine, FSM）管理。FSM 确保机器人在任何时刻都处于明确定义的状态，且所有状态转换都经过安全校验。

3.4.1 形式化定义

FSM 形式化定义为五元组：

$$\text{FSM} = (S, \Sigma, \delta, s_0, F) \quad (3.12)$$

其中各元素定义如下：

$$S = \{\text{Zero, Grounded, Standing, Walking, Transitioning}\} \quad (3.13)$$

$$\Sigma = \{\text{init, standUp, sitDown, walk, stop, fault}\} \quad (3.14)$$

$$s_0 = \text{Zero} \quad (3.15)$$

$\delta : S \times \Sigma \rightarrow S$ 为状态转移函数， F 为终态集合（在持续运行系统中通常为空集）。

3.4.2 状态转移表

当前状态	输入事件	下一状态
Zero	init	Grounded
Grounded	standUp	Transitioning $\rightarrow$ Standing
Standing	walk	Walking
Standing	sitDown	Transitioning $\rightarrow$ Grounded
Walking	stop	Standing
Walking	sitDown	Transitioning $\rightarrow$ Grounded
任意状态	fault	Grounded（经由安全坐下）

3.4.3 安全不变量

FSM 设计中最重要安全不变量：

从任意状态出发，收到 `fault` 信号后，系统必须经由安全坐下动画回到 `Grounded` 状态。

这意味着即使在行走过程中发生低电压告警、通信丢失或软件异常，机器人也会优雅地坐下而非直接断电跌倒。这一设计在代码中由 `Brain` 的 FSM 实现保证（见 `han_dog_brain/cms/`）。

3.5 控制频率与实时性

ORIX 的主控制循环以 50 Hz 固定频率运行，即每个控制周期为：

$$T = \frac{1}{50 \text{ Hz}} = 20 \text{ ms} \quad (3.16)$$

3.5.1 单周期流水线

每个 20 ms 周期内，系统依次执行以下步骤：

1. **传感器读取**：从 IMU 和关节编码器获取最新数据
2. **观测构建**：将传感器数据组装为策略网络所需的观测向量
3. **神经网络推理**：策略网络前向传播，输出关节动作
4. **动作后处理**：应用 `actionScale` 缩放和安全限幅
5. **电机指令下发**：通过 CAN 总线发送 PD 位置指令

3.5.2 时间预算

典型的时间分配如下：

阶段	耗时
神经网络推理（ONNX Runtime）	< 1 ms
传感器读取 + 观测构建 + 动作后处理	< 1 ms
总流水线	< 2 ms
事件循环空闲时间	~ 18 ms

3.5.3 为什么单线程事件循环足够

ORIX 主控程序采用 Dart 单线程异步事件循环模型。50 Hz 控制循环的关键路径仅占用约 2 ms，远小于 20 ms 的周期。剩余 18 ms 可用于处理 gRPC 请求、日志写入和状态广播等异步任务。

单线程模型的优势在于：

- 无线程同步开销（无锁、无竞态条件）
- 确定性的执行顺序，便于调试和分析
- 更低的延迟抖动（jitter），提高控制精度

只要控制路径上的操作全部是非阻塞的（ORIX 通过异步 I/O 和 CAN 总线非阻塞写入保证这一点），单线程模型完全能够满足 50 Hz 的实时性要求。

Chapter 4

坐标系与关节

本章导读

本章用于统一方向、编号和姿态语言，避免多人协作时出现“左右相反”“关节号对不上”“旋转方向理解不一致”这类低级但高成本的问题。

阅读建议：开始写控制代码前，至少应熟悉机体坐标系、四条腿的编号分组以及 `walk()` 的三个速度轴向定义。

本文档定义 ORIX 机器人的坐标系约定、16 个关节的编号规则、运动范围和典型姿态。

4.1 机体坐标系

ORIX 使用右手坐标系，原点位于机身几何中心：

机体坐标系采用右手定则：X 轴指向机器人前方，Y 轴指向机器人左侧，Z 轴垂直向上。原点位于机身几何中心。

轴	正方向	说明
X	前方 (Forward)	机器人头部方向
Y	左方 (Left)	机器人左侧方向
Z	上方 (Up)	垂直向上

4.1.1 正面视图

从正面观察机器人：Z 轴向上，Y 轴向右为负、向左为正。左上方为 FL 腿，右上方为 FR 腿，左下方为 RL 腿，右下方为 RR 腿。机身 (body) 位于中央。

4.1.2 侧面视图（右侧）

从右侧观察机器人：Z 轴向上，X 轴指向前方（即机器人前进方向）。机身上方连接两条腿，每条腿从上到下依次为 Hip、Thigh、Calf、Foot 关节，Foot（脚轮）接触地面。后方为后腿（RR），前方为前腿（FR）。

4.1.3 walk() 指令与坐标系的关系

理解行走指令参数与坐标轴的对应关系，是正确控制机器狗运动方向的前提。下表列出了三个速度参数的物理含义。

参数	正值	负值	对应轴
vx	前进	后退	+X / -X
vy	向左	向右	+Y / -Y
vyaw	逆时针	顺时针	绕 +Z 轴

4.2 关节编号总表

以下是 ORIX 全部 16 个关节的完整参数表，包括编号、URDF 名称、运动范围和最大力矩。编写底层控制或进行运动学分析时需要频繁参考此表。ORIX 共 16 个自由度，按腿分组编号：

索引	名称	腿	关节类型	URDF 名称	范围 (rad)	正方向	最大力矩
0	FR Hip	前右	髋 (横滚)	fr_hip_joint	-0.41 ~ +0.41	外展	50 Nm
1	FR Thigh	前右	大腿 (俯仰)	fr_thigh_joint	-3.14 ~ 0	后摆	50 Nm
2	FR Calf	前右	小腿 (俯仰)	fr_calf_joint	0 ~ +3.14	伸展	50 Nm
3	FR Foot	前右	脚轮 (连续)	fr_foot_joint	连续旋转	—	30 Nm
4	FL Hip	前左	髋 (横滚)	fl_hip_joint	-0.41 ~ +0.41	外展	50 Nm
5	FL Thigh	前左	大腿 (俯仰)	fl_thigh_joint	0 ~ +3.14	前摆	50 Nm
6	FL Calf	前左	小腿 (俯仰)	fl_calf_joint	-3.14 ~ 0	收缩	50 Nm
7	FL Foot	前左	脚轮 (连续)	fl_foot_joint	连续旋转	—	30 Nm
8	RR Hip	后右	髋 (横滚)	rr_hip_joint	-0.41 ~ +0.41	外展	50 Nm
9	RR Thigh	后右	大腿 (俯仰)	rr_thigh_joint	0 ~ +3.14	前摆	50 Nm
10	RR Calf	后右	小腿 (俯仰)	rr_calf_joint	-6.28 ~ 0	收缩	50 Nm
11	RR Foot	后右	脚轮 (连续)	rr_foot_joint	连续旋转	—	30 Nm
12	RL Hip	后左	髋 (横滚)	rl_hip_joint	-0.41 ~ +0.41	外展	50 Nm
13	RL Thigh	后左	大腿 (俯仰)	rl_thigh_joint	-3.14 ~ 0	后摆	50 Nm
14	RL Calf	后左	小腿 (俯仰)	rl_calf_joint	0 ~ +6.28	伸展	50 Nm
15	RL Foot	后左	脚轮 (连续)	rl_foot_joint	连续旋转	—	30 Nm

4.3 关节分组

4.3.1 按腿分组

在诊断特定腿部问题或编写单腿测试程序时，按腿分组的索引非常实用。

腿	关节索引
---	------

前右 (FR)	[0, 1, 2, 3]
前左 (FL)	[4, 5, 6, 7]
后右 (RR)	[8, 9, 10, 11]
后左 (RL)	[12, 13, 14, 15]

4.3.2 按关节类型分组

按类型分组便于批量调整同类关节的 PD 参数或进行统一的故障检测。

关节类型	索引	功能
所有 Hip	[0, 4, 8, 12]	髋关节，控制腿的内外摆动
所有 Thigh	[1, 5, 9, 13]	大腿关节，控制腿的前后摆动
所有 Calf	[2, 6, 10, 14]	小腿关节，控制膝盖弯曲
所有 Foot	[3, 7, 11, 15]	脚轮，控制轮式移动

4.3.3 俯瞰图 (Top View)

从俯瞰角度看，X 轴正方向朝前 (Head)，X 轴负方向朝后 (Tail)，Y 轴正方向朝左，Y 轴负方向朝右。四条腿的关节索引分布为：左前 FL[4-7] 位于左前方，右前 FR[0-3] 位于右前方，左后 RL[12-15] 位于左后方，右后 RR[8-11] 位于右后方。机身中心为坐标原点。

4.3.4 侧面图——单腿结构

以前右腿 (FR) 为例，单腿从躯干到地面的关节链结构如下：

1. [0] **Hip** (髋关节)：连接躯干，绕 X 轴旋转 (横滚)，控制腿的内外摆动。
2. [1] **Thigh** (大腿关节)：连接上臂段，绕 Y 轴旋转 (俯仰)，控制腿的前后摆动。
3. [2] **Calf** (小腿关节)：连接下臂段，绕 Y 轴旋转 (俯仰)，控制膝盖弯曲。
4. [3] **Foot** (脚轮)：末端执行器，连续旋转驱动轮毂，脚轮触地。

4.4 典型姿态

4.4.1 站立姿态 (Standing Pose)

站立姿态定义了机器人正常直立时所有关节的目标角度。RL 策略的输出会在此姿态基础上叠加偏移量，驱动步态运动。以下是各关节的目标角度：

```

1 standing_pose = [
2     0.0, -0.65,  1.5,  0,    # FR: hip, thigh, calf, foot
3     0.0,  0.65, -1.5,  0,    # FL: hip, thigh, calf, foot
4     0.0, -0.65,  1.5,  0,    # RR: hip, thigh, calf, foot
5     0.0,  0.65, -1.5,  0,    # RL: hip, thigh, calf, foot

```

6]

各关节含义：

关节	FR 值	含义
Hip	0.0 rad (0°)	髋关节居中
Thigh	-0.65 rad (-37.2°)	大腿向后摆约 37 度
Calf	1.5 rad (85.9°)	小腿弯曲约 86 度
Foot	0 rad	脚轮不转

注意：注意左右对称：FR 和 RR 的符号相同，FL 和 RL 的符号相同（镜像关系）。

4.4.2 坐下姿态 (Sitting Pose)

坐下姿态即电机零位，所有关节角度为 0：

```
1 sitting_pose = [0, 0, 0, 0, # FR
2                 0, 0, 0, 0, # FL
3                 0, 0, 0, 0, # RR
4                 0, 0, 0, 0] # RL
```

这是机器人趴在地面上的姿态，也是电机标零后的参考位。

4.4.3 姿态可视化

站立姿态（Standing）时，机器人四腿伸展支撑躯干离地，重心较高，脚轮接触地面。坐下姿态（Sitting / Zero）时，所有关节归零，腿部折叠收拢，躯干贴近地面，这是最稳定的安全状态。

4.5 PD 控制参数

每个关节的运动由 PD 控制器驱动：

$$\tau = K_p \cdot (q_{\text{target}} - q_{\text{current}}) + K_d \cdot (\dot{q}_{\text{target}} - \dot{q}_{\text{current}})$$

4.5.1 推理阶段（行走时）

行走时 RL 策略以 50Hz 输出动作向量，经过动作缩放和 PD 控制器计算后转化为电机电力矩指令。以下是各关节类型的控制参数。

索引	关节	Kp	Kd	actionScale
0, 4, 8, 12	Hip	80	10	0.15

索引	关节	Kp	Kd	actionScale
1, 5, 9, 13	Thigh	100	10	0.25
2, 6, 10, 14	Calf	120	15	0.25
3, 7, 11, 15	Foot	0	1	5.0

`actionScale` 是 RL 策略输出到实际弧度的缩放系数。策略输出值经过以下变换得到目标关节角度：

$$q_{\text{target}} = \text{action_output} \times \text{actionScale} + \text{standingPose}$$

4.5.2 站立姿态

站立姿态（Standing Pose）是机器狗正常直立时所有关节的目标角度。站起过程会从坐姿线性插值到站立姿态，约 3 秒完成。站立姿态和 PD 增益参数均通过 Profile JSON 文件配置，用户可以根据自己的需要调整。

4.5.3 PD 控制接口

机器狗的关节控制基于 PD（比例-微分）位置控制。每个关节独立接受 Kp（比例增益）和 Kd（微分增益）参数，不同的增益组合会产生不同的运动效果：Kp 越大关节越“硬”，响应越快但可能震荡；Kd 越大阻尼越强，运动越平滑但响应变慢。

PD 参数通过 Profile JSON 文件配置，SDK 提供以下接口查看和切换策略：

```

1 # 查看当前策略（含 PD 增益配置）
2 profile = dog.get_profile()
3 print(profile.current)      # 当前策略名称
4 print(profile.available)    # 所有可用策略
5
6 # 切换到不同的 PD 参数组合（机器人须在着地状态）
7 dog.switch_profile("my_custom_profile")

```

Profile JSON 文件中的 PD 相关字段：

字段	说明
<code>inferKp / inferKd</code>	行走推理时的 PD 增益（16 个值，每关节独立）
<code>standUpKp / standUpKd</code>	站起过渡时的 PD 增益
<code>sitDownKp / sitDownKd</code>	坐下过渡时的 PD 增益
<code>actionScale</code>	动作缩放系数（4 个值，按关节类型）

详细的 JSON 格式规范请参考 `profile_schema.json`。

4.6 编程中的关节操作

4.6.1 读取关节数据

通过 `listen_joint()` 流式接口可以实时获取每个关节的位置、速度和力矩数据，适合用于数据记录和运动分析。

```
1 # 实时流式读取
2 for joint in dog.listen_joint():
3     print(f"关节 {joint.id}: "
4           f"位置={joint.position:.3f}rad "
5           f"速度={joint.velocity:.3f}rad/s "
6           f"力矩={joint.torque:.2f}Nm")
```

4.6.2 诊断单条腿

当怀疑某条腿存在硬件问题时，可以单独查询该腿四个电机的状态信息。

```
1 # 检查前右腿 (FR) 的 4 个电机
2 motors = dog.get_motor_status()
3 fr_motors = [m for m in motors if m.id in [0, 1, 2, 3]]
4 for m in fr_motors:
5     print(f"  关节{m.id}: online={m.online} temp={m.temperature:.1f}C")
```

4.6.3 清除单腿故障

如果只有特定腿的电机报告故障，可以仅清除该腿的故障码，而不影响其他正常工作的电机。

```
1 # 只清除前右腿的电机故障
2 dog.clear_motor_fault(joint_ids=[0, 1, 2, 3])
```


Chapter 5

状态机与运动控制

本章导读

本章解释“什么状态下什么指令才会生效”，是理解运动控制、排查动作不响应以及设计安全流程的核心章节。

阅读建议：如果您刚开始接触 SDK，建议先读“状态机总览”“状态转换详解”和“典型使用流程”，再回头查各类异常和边界条件。

本文档详细说明 ORIX 机器人的有限状态机 (FSM)、状态转换规则、运动控制接口和安全机制。

5.1 状态机总览

ORIX 的运动控制由一个严格的有限状态机管理。所有 SDK 指令必须遵循状态机规则，非法的状态转换会被拒绝。

5.1.1 状态图

ORIX 的有限状态机包含五个状态，以下是完整的状态转换路径：

起始状态	触发条件	中间状态	目标状态
Zero	系统初始化 Init()	–	Grounded
Grounded	stand_up()	Transitioning (StandUp)	Standing
Standing	walk(vx, vy, vyaw)	–	Walking
Walking	walk() 停步或 Idle	Transitioning (StandUp)	Standing
Standing	sit_down()	Transitioning (SitDown)	Grounded
Walking	sit_down()	先 StandUp 再 SitDown	Grounded
任意运动状态	Fault 故障	自动过渡	Grounded（安全坐下）

状态机的核心原则是：所有运动指令必须遵循状态转换顺序，Transitioning（过渡中）状态下拒绝新的运动指令，Fault 触发时系统会自动执行安全着陆。

5.1.2 五个状态

状态	SDK 返回值	说明
Zero	"zero"	上电初始态，FSM 尚未初始化。不接受任何运动指令。
Grounded	"grounded"	坐姿/趴地。安全状态，电机可使能但不运动。
Standing	"standing"	直立站稳。可以接收 <code>walk()</code> 或 <code>sit_down()</code> 指令。
Walking	"walking"	行走中。RL 策略以 50Hz 推理驱动步态。
Transitioning	"transitioning"	过渡中（站起/坐下/动作播放）。拒绝新的运动指令。

5.2 状态转换详解

5.2.1 Zero → Grounded

这是系统上电后的第一个自动状态转换，无需用户干预。

项目	说明
触发	系统自动初始化 (Init)
耗时	即时
用户操作	无需操作，上电后自动完成

上电启动 `brainstem` 服务后，FSM 自动从 Zero 进入 Grounded。

5.2.2 Grounded → Standing（站起）

站起过程是用户主动触发的第一个运动动作，需要约 3 秒完成平滑过渡。

项目	说明
触发	<code>dog.stand_up()</code>
中间状态	Transitioning (StandUp)
耗时	$150 \text{ ticks} \times 20\text{ms} = \mathbf{3 \text{ 秒}}$
过程	从当前姿态线性插值到 <code>standingPose</code>
PD 参数	由 Profile 配置决定（过渡阶段使用较高刚度，确保平稳）

站起过程是一段 150 帧的线性插值动画：

站起过程分为 150 帧，按线性插值平滑过渡：第 0 帧为当前坐姿 (`sittingPose`)，第 75 帧为中间过渡态（50% 位置），第 150 帧为完全站立姿态 (`standingPose`)。

5.2.3 Standing → Walking（开始行走）

从站立到行走的切换是即时的，RL 步态策略会立即接管关节控制。

项目	说明
触发	<code>dog.walk(vx, vy, vyaw)</code>
耗时	即时切换
过程	RL 策略接管，以 50Hz 推理输出关节指令

5.2.4 Walking 中更新方向

行走过程中可以随时更新速度方向，无需停步再重新发送指令。新的方向在下一个控制周期即刻生效。

项目	说明
触发	再次调用 <code>dog.walk(vx, vy, vyaw)</code>
耗时	即时生效（下一帧）
过程	仅更新方向向量，不切换状态

5.2.5 Walking → Standing（停步）

停步操作会让机器狗从行走状态平滑过渡回站立姿态。

项目	说明
触发	<code>dog.walk()</code> 无参数，或发送全零后超时自动触发 Idle
中间状态	Transitioning (StandUp)，从当前行走姿态插值到 <code>standingPose</code>
耗时	~3 秒

5.2.6 Standing → Grounded（坐下）

坐下是结束运动任务的标准操作，也是安全关机流程的必要步骤。

项目	说明
触发	<code>dog.sit_down()</code>
中间状态	Transitioning (SitDown)
耗时	$150 \text{ ticks} \times 20\text{ms} = \mathbf{3 \text{ 秒}}$
过程	从 <code>standingPose</code> 线性插值到 <code>sittingPose</code> (全零)

5.2.7 Walking → Grounded（行走中直接坐下）

在行走状态下也可以直接调用坐下指令，系统会自动执行“先站稳、再坐下”的复合过渡。

项目	说明
触发	在 Walking 状态调用 <code>dog.sit_down()</code>
过程	复合过渡：先自动站稳 (StandUp)，再坐下 (SitDown)
耗时	~6 秒（两段过渡串联）

5.3 非法操作处理

ORIX 的状态机设计优先保证安全性。高层 SDK API 会抛出 `InvalidStateError` 异常（从 `grpc.FAILED_PRECONDITION` 映射而来）。如果您直接使用底层 stub 绕过 SDK，服务端可能静默忽略不合法的状态转换。推荐始终使用 `OrixClient` 并处理 `InvalidStateError`：

当前状态	发送的指令	结果
Grounded	<code>walk()</code>	忽略。必须先 <code>stand_up()</code>
Grounded	<code>sit_down()</code>	忽略。已在坐姿
Standing	<code>stand_up()</code>	忽略。已站立
Walking	<code>stand_up()</code>	触发从行走过渡到站立
Transitioning	任何运动指令	拒绝。过渡期间不接受新指令
Zero	任何运动指令	忽略。FSM 未初始化

最佳实践：在发送指令前先查询状态，确认状态机处于正确状态。

```

1 state = dog.get_state()
2 if state == "standing":
3     dog.walk(vx=0.3)
4 elif state == "grounded":
5     dog.stand_up()
```

5.4 walk() 速度约定

5.4.1 参数说明

`walk()` 方法接受三个归一化速度参数，分别控制前后、左右和旋转方向。所有参数默认为 0，即原地停步。

阅读本节时，请把这三个参数理解为“控制意图”而不是“物理量”。它们表达的是当前策略允许范围内的归一化指令，因此安全调试时应优先从小值开始，例如 0.1 到 0.2，确认方向正确后再逐步加大。

```

1 dog.walk(vx=0.0, vy=0.0, vyaw=0.0)
```

参数	范围	正值	负值	零值
vx	[-1.0, 1.0]	前进 (+X)	后退 (-X)	不动
vy	[-1.0, 1.0]	向左 (+Y)	向右 (-Y)	不动
vyaw	[-1.0, 1.0]	逆时针 (+Z)	顺时针 (-Z)	不转

三个参数均为归一化速度，1.0 对应最大速度，具体物理速度由 RL 策略决定。

5.4.2 常见用法

以下代码片段展示了最常用的运动指令组合，包括直线行走、横移、旋转和复合运动。

```
1 # 前进
2 dog.walk(vx=0.5)
3
4 # 后退
5 dog.walk(vx=-0.3)
6
7 # 向左平移
8 dog.walk(vy=0.3)
9
10 # 原地逆时针转
11 dog.walk(vyaw=0.3)
12
13 # 边走边转（前进 + 左转）
14 dog.walk(vx=0.3, vyaw=0.2)
15
16 # 停步（发送零速度）
17 dog.walk()
```

5.4.3 运动方向参考图

运动方向与参数的对应关系： $vx > 0$ 为前进， $vx < 0$ 为后退； $vy > 0$ 为向左平移， $vy < 0$ 为向右平移； $vyaw > 0$ 为逆时针旋转（CCW）， $vyaw < 0$ 为顺时针旋转（CW）。三个参数可以组合使用，实现边走边转等复合运动。

5.5 电机使能/禁用生命周期

5.5.1 完整生命周期

理解电机的使能/禁用生命周期对于安全操作至关重要。电机的生命周期如下：上电后电机默认处于禁用状态。调用 `enable()` 使能电机后，可以执行 `stand_up()`、`walk()`、`sit_down()`

等运动操作。在任何使能状态下调用 `disable()` 都会立即释放所有电机力矩，电机回到禁用状态。最终下电关闭电源。

5.5.2 关键规则

1. 上电后电机默认禁用。必须调用 `enable()` 才能运动。
2. `disable()` 随时可调用。会立即释放所有电机力矩（机器人会瘫软）。
3. 先坐下再禁用。建议流程：`sit_down()` → 等待 Grounded → `disable()`。
4. 低压自动禁用。电压低于 18V 时，系统会先坐下再禁用，不会直接断电。

5.5.3 安全的关机流程

以下代码展示了如何从任意运动状态安全地关闭机器狗。核心原则是：先确认坐稳，再禁用电机。

```
1 import time
2
3 # 确保从任何状态安全关闭
4 state = dog.get_state()
5
6 if state == "walking":
7     dog.sit_down()      # Walking -> StandUp -> SitDown (复合)
8     time.sleep(7)       # 等待两段过渡完成
9 elif state == "standing":
10    dog.sit_down()
11    time.sleep(4)
12 elif state == "transitioning":
13    time.sleep(4)        # 等待当前过渡完成
14
15 # 确认已坐下
16 assert dog.get_state() == "grounded"
17
18 # 安全禁用
19 dog.disable()
```

5.6 控制权仲裁 (ControlArbiter)

5.6.1 优先级机制

ORIX 支持多个控制源同时存在，通过 `ControlArbiter` 仲裁控制权：

控制权优先级从高到低依次为：遥控器（最高优先级）、SDK gRPC（较低优先级）。遥控器活跃时，SDK 的指令将被忽略。遥控器无操作超过 3 秒后，控制权自动释放给 SDK。

5.6.2 仲裁规则

场景	行为
SDK 控制中，遥控器摇杆动	SDK 被立即抢占，遥控器接管
遥控器控制中，SDK 发指令	SDK 指令被忽略
遥控器松手不动	3 秒超时后自动释放，SDK 恢复控制权
SDK 控制中，遥控器关机	SDK 保持控制权，不受影响

5.6.3 超时参数

参数	默认值	环境变量	说明
arbiterTimeout	3 秒	HAN_DOG_ARBITER_TIMEOUT	遥控器释放控制权的超时时间

5.6.4 SDK 开发建议

1. 首次调试时关闭遥控器，避免控制权冲突
2. 保留遥控器作为安全手段，紧急情况下随时接管
3. 不要轮询状态来检测控制权。当 SDK 指令与当前状态不匹配时，OrixClient 会抛出 InvalidStateError 异常。遥控器持有控制权时的 SDK 指令在仲裁层被拒绝，这种情况 SDK 不会报错（非状态机错误）。

5.7 故障处理

5.7.1 Fault 状态转换

当系统检测到异常时，会触发 Fault 动作。不同状态下的 Fault 行为：

当前状态	Fault 行为
Grounded	忽略（已在安全状态）
Standing	忽略（已在安全状态）
Walking	自动过渡到 StandUp（先站稳）
Transitioning (StandUp)	中止站起，转为 SitDown
Transitioning (SitDown)	强制进入 Grounded（避免死循环）

5.7.2 电机故障清除

```
1 # 查看电机状态
2 motors = dog.get_motor_status()
```

```
3 for m in motors:
4     if m.errors:
5         print(f"关节 {m.id} 故障: {m.errors}")
6
7 # 清除全部故障
8 dog.clear_motor_fault()
9
10 # 清除指定关节的故障
11 dog.clear_motor_fault(joint_ids=[0, 1, 2])
```

5.7.3 低压保护

当任一电机总线电压低于 18V 时触发：

低压保护触发后，系统依次执行以下步骤：（1）检测到低压（低于 18V）→（2）自动发送坐下指令 →（3）等待机器人进入 *Grounded* 状态 →（4）禁用全部电机。

低压保护是**不可逆的**（单次触发后标记 `lowVoltageTriggered`），需要重新上电才能恢复。

5.8 策略切换

ORIX 支持多种运动策略（Profile），每种策略包含不同的 ONNX 模型和 PD 参数。

5.8.1 查询和切换

```
1 # 查看当前策略
2 profile = dog.get_profile()
3 print(f"当前: {profile.current}")
4 print(f"可用: {profile.available}")
5
6 # 切换策略（必须在 Grounded 状态）
7 dog.switch_profile("mini")
```

5.8.2 切换规则

- 只能在 **Grounded** 状态下切换策略
- 切换后会重新加载 ONNX 模型
- 新策略的 PD 参数和动作缩放会立即生效

5.9 典型使用流程

5.9.1 标准操作序列

```
1 import time
2 from brainstem_sdk import OrixClient
3
4 # 连接
5 dog = OrixClient("192.168.66.190")
6
7 # 1. 诊断检查
8 state = dog.get_state()
9 print(f"初始状态: {state}")          # 应为 "grounded"
10
11 voltages = dog.get_voltage()
12 avg_v = sum(voltages) / len(voltages) if voltages else 0
13 print(f"电池电压: {avg_v:.1f}V")    # 应 > 18V
14
15 motors = dog.get_motor_status()
16 offline = [m.id for m in motors if not m.online]
17 if offline:
18     print(f"警告: 关节 {offline} 离线")
19
20 # 2. 使能电机
21 dog.enable()
22
23 # 3. 站起 (3 秒过渡)
24 dog.stand_up()
25 time.sleep(4)
26 assert dog.get_state() == "standing"
27
28 # 4. 行走
29 dog.walk(vx=0.3)                    # 前进
30 time.sleep(3)
31
32 dog.walk(vx=0.3, vyaw=0.2)          # 前进 + 左转
33 time.sleep(2)
34
35 dog.walk()                          # 停步
36 time.sleep(1)
37
38 # 5. 坐下 (3 秒过渡)
39 dog.sit_down()
40 time.sleep(4)
41 assert dog.get_state() == "grounded"
```

```
42
43 # 6. 禁用电机
44 dog.disable()
45
46 # 7. 断开连接
47 dog.close()
```

5.9.2 实时数据监控

```
1 import threading
2 import time
3 from brainstem_sdk import OrixClient
4
5 dog = OrixClient("192.168.66.190")
6
7 # 在后台线程中监听 IMU
8 def monitor_imu():
9     for imu in dog.listen_imu():
10         gx, gy, gz = imu.gyroscope.x, imu.gyroscope.y, imu.gyroscope.z
11         print(f"\rIMU: gx={gx:+.2f} gy={gy:+.2f} gz={gz:+.2f}", end="")
12
13 imu_thread = threading.Thread(target=monitor_imu, daemon=True)
14 imu_thread.start()
15
16 # 在后台线程中监听状态变化
17 def monitor_state():
18     for state in dog.listen_state():
19         print(f"\n[STATE] -> {state}")
20
21 state_thread = threading.Thread(target=monitor_state, daemon=True)
22 state_thread.start()
23
24 # 主线程执行操作
25 dog.enable()
26 dog.stand_up()
27 time.sleep(4)
28 dog.walk(vx=0.3)
29 time.sleep(5)
30 dog.sit_down()
31 time.sleep(4)
32 dog.disable()
33 dog.close()
```

5.9.3 安全包装器

建议在生产代码中使用 try/finally 确保安全退出：

```
1 import time
2 from brainstem_sdk import OrixClient
3
4 dog = OrixClient("192.168.66.190")
5
6 try:
7     dog.enable()
8     dog.stand_up()
9     time.sleep(4)
10
11     # ... 您的业务逻辑 ...
12     dog.walk(vx=0.3)
13     time.sleep(5)
14
15 finally:
16     # 无论发生什么异常，都确保安全坐下并禁用
17     try:
18         dog.sit_down()
19         time.sleep(5)
20     except Exception:
21         pass
22     try:
23         dog.disable()
24     except Exception:
25         pass
26     dog.close()
```

5.10 关键参数速查

参数	值	说明
控制频率	50 Hz (20ms)	主控制循环和推理频率
gRPC 端口	13145	SDK 连接端口
站起耗时	3 秒 (150 ticks)	Grounded → Standing
坐下耗时	3 秒 (150 ticks)	Standing → Grounded
仲裁超时	3 秒	遥控器释放控制权的等待时间
低压阈值	18V	低于此值触发保护坐下

参数	值	说明
启动超时	10 秒	FSM 等待 Grounded 的超时
关机超时	8 秒	安全关机等待超时
RPC 超时	5 秒	SDK 默认 RPC 超时

Chapter 6

实用开发指南

本章导读

本章汇集了在实际开发中经常遇到的场景和最佳实践。包括电池管理、数据记录、远程部署、集成 OpenCV 视觉、自定义 Profile 和多机器人管理等实用模式。每个场景都给出可直接使用的代码模板。

阅读建议：本章不涉及新的 API，而是展示如何组合已有 API 解决真实问题。初次阅读可以略读，实际开发时作为速查参考。

6.1 电池管理

机器狗采用 7 串锂电池，正常工作电压约 24V，低压保护阈值 18V。在长时间运行的应用中，建议实现主动电池监控而不是依赖低压保护被动触发。

6.1.1 电池电量估算

通过电压近似估算电池电量（线性估算，实际锂电放电曲线非线性，仅作参考）：

```
1 def estimate_battery_level(dog):
2     """估算电池电量百分比（近似值）"""
3     voltages = dog.get_voltage()
4     valid = [v for v in voltages if v > 0]
5     if not valid:
6         return None
7     avg_v = sum(valid) / len(valid)
8     # 线性插值：29.4V=100%，18V=0%
9     pct = max(0, min(100, (avg_v - 18) / (29.4 - 18) * 100))
10    return pct, avg_v
```

6.1.2 周期性电池监控

在长时间运行的任务中，建议每 30 秒检查一次电池，电量低于 30% 时主动坐下并提醒用户：

```
1 import time
```

```
2 from brainstem_sdk import OrixClient
3
4 def monitor_battery_loop(dog, interval=30):
5     while True:
6         pct, v = estimate_battery_level(dog)
7         if v is None:
8             time.sleep(interval)
9             continue
10        print(f"电量: {pct:.0f}% ({v:.1f}V)")
11        if pct < 30:
12            print("低电量, 主动收工...")
13            dog.sit_down()
14            dog.disable()
15            break
16        time.sleep(interval)
```

6.2 数据记录与采集

机器狗的 IMU 和关节数据是进行算法研究和系统调试的重要数据源。SDK 的流式接口可以方便地记录这些数据到文件。

6.2.1 记录 IMU 数据到 CSV

```
1 import csv
2 from brainstem_sdk import OrixClient
3
4 def record_imu_to_csv(duration_sec, filename):
5     dog = OrixClient("192.168.66.190")
6     with open(filename, "w", newline="") as f:
7         writer = csv.writer(f)
8         writer.writerow(["time_ms", "gx", "gy", "gz",
9                          "qw", "qx", "qy", "qz"])
10        count = 0
11        target = duration_sec * 50 # 50 Hz
12        for imu in dog.listen_imu():
13            writer.writerow([
14                imu.timestamp_ms,
15                imu.gyroscope.x, imu.gyroscope.y, imu.gyroscope.z,
16                imu.quaternion.w, imu.quaternion.x,
17                imu.quaternion.y, imu.quaternion.z,
18            ])
19            count += 1
```

```
20         if count >= target:
21             break
22     dog.close()
23
24 # 记录 10 秒 IMU 数据
25 record_imu_to_csv(10, "imu_log.csv")
```

6.2.2 同时记录 IMU 和关节数据

使用多线程同时订阅多个数据流，主线程负责写文件协调：

```
1  import threading
2  from queue import Queue
3
4  def record_all_sensors(dog, duration_sec, prefix="log"):
5      imu_q = Queue()
6      joint_q = Queue()
7      stop_event = threading.Event()
8
9      def imu_worker():
10         for imu in dog.listen_imu():
11             if stop_event.is_set():
12                 break
13             imu_q.put(imu)
14
15     def joint_worker():
16         for j in dog.listen_joint():
17             if stop_event.is_set():
18                 break
19             joint_q.put(j)
20
21     t1 = threading.Thread(target=imu_worker, daemon=True)
22     t2 = threading.Thread(target=joint_worker, daemon=True)
23     t1.start()
24     t2.start()
25
26     time.sleep(duration_sec)
27     stop_event.set()
28
29     # 保存队列内容到文件
30     # ... (写 CSV 逻辑)
```

6.3 远程部署

大多数情况下您在开发电脑上运行 Python 代码通过 gRPC 远程控制机器狗。但有时需要将代码部署到机器狗本体上运行（例如减少网络延迟、脱离开发机运行）。

6.3.1 SSH 部署 workflow

将本地脚本上传到机器狗并运行：

```
1 # 上传脚本到机器狗
2 scp my_script.py sunrise@192.168.66.190:~/scripts/
3
4 # SSH 远程执行
5 ssh sunrise@192.168.66.190 "cd scripts && python3 my_script.py"
6
7 # 长期后台运行
8 ssh sunrise@192.168.66.190 "nohup python3 scripts/my_script.py > log.txt
   2>&1 &"
```

在机器狗本地运行时，SDK 的连接地址使用 localhost：

```
1 # 本地运行（在机器狗上）
2 dog = OrixClient("localhost")
3
4 # 远程运行（在开发机上）
5 dog = OrixClient("192.168.66.190")
```

6.4 集成计算机视觉

结合 OpenCV 可以实现基于视觉的控制，例如目标跟踪、障碍物检测等。以下是一个最简单的颜色追踪示例：

```
1 import cv2
2 import numpy as np
3 from brainstem_sdk import OrixClient, OrixCamera
4
5 def follow_red_ball():
6     dog = OrixClient("192.168.66.190")
7     cam = OrixCamera()
8     cam.open()
9
10    dog.enable()
11    dog.stand_up()
```



```
12
13     try:
14         while True:
15             frame = cam.read()
16             if frame is None:
17                 continue
18
19             # 检测红色区域
20             hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
21             mask = cv2.inRange(hsv, (0, 100, 100), (10, 255, 255))
22             moments = cv2.moments(mask)
23
24             if moments["m00"] > 500:
25                 # 红色物体中心 x 坐标
26                 cx = int(moments["m10"] / moments["m00"])
27                 frame_center = frame.shape[1] // 2
28                 # 根据偏差控制转向
29                 turn = -(cx - frame_center) / frame_center * 0.3
30                 dog.walk(vx=0.2, vyaw=turn)
31             else:
32                 dog.walk() # 停止
33
34             if cv2.waitKey(1) == ord('q'):
35                 break
36         finally:
37             dog.walk()
38             dog.sit_down()
39             dog.disable()
40             cam.close()
41             dog.close()
```

6.5 自定义 Profile

Profile 是机器狗的运动参数集合，用户可以通过自定义 Profile 调整 PD 增益、站立姿态和 ONNX 模型路径。自定义 Profile 的典型用途包括：训练自己的 RL 策略、调参使机器狗更稳定、适配不同地形。

6.5.1 Profile 文件结构

Profile 是 JSON 文件，存放在机器狗的 `profiles/` 目录下。以下是一个完整示例的骨架：

```
{
```

```
"name": "my_custom",
"description": "自定义策略",
"modelPath": "model/my_policy.onnx",
"standingPose": [0.0, -0.65, 1.5, 0.0, ...],
"sittingPose": [0.0, 0.0, 0.0, 0.0, ...],
"inferKp": [...], "inferKd": [...],
"standUpKp": [...], "standUpKd": [...],
"sitDownKp": [...], "sitDownKd": [...],
"standUpCounts": 150,
"sitDownCounts": 150,
"imuGyroscopeScale": 0.25,
"jointVelocityScale": [0.05, 0.05, 0.05, 0.05],
"actionScale": [0.125, 0.25, 0.25, 5.0]
}
```

完整字段说明参见 SDK 安装目录下的 `profile_schema.json`。

6.5.2 部署自定义 Profile

```
1 # 上传到机器狗
2 scp my_custom.json sunrise@192.168.66.190:~/profiles/
3
4 # 控制系统每 30 秒自动扫描 profiles/ 目录，无需重启
5 # 上传后在 SDK 中即可切换
```

```
1 # 切换到自定义策略
2 dog.switch_profile("my_custom")
3 print(f"当前策略: {dog.get_profile().current}")
```

6.6 开发建议与最佳实践

- 始终使用 `try/finally` — 确保异常时机器狗能安全收工（坐下 + 禁用电机）
- 先查状态再发指令 — 状态不匹配的指令会被静默忽略，先 `get_state()` 可以避免误判
- 流式接口用迭代器 — `listen_imu()` 等返回迭代器，用 `for` 循环消费，不要一次性 `list()`
- 网络延迟补偿 — 通过 Wi-Fi 连接时有 5-20ms 延迟，对需要高实时性的控制任务建议部署到机器狗本地
- 日志分级 — 开发时用 `INFO` 查看状态，调试时用 `DEBUG`，生产时用 `WARNING`

- **测试先仿真后真机** — 大算机器人提供 MuJoCo 仿真环境，建议先在仿真中验证逻辑再部署到真机
- **电池健康** — 经常使用到低压的电池寿命会下降，建议保持在 20%–80% 之间充放电

Chapter 7

传感器数据

本章导读

本章重点说明各类传感器数据的含义、频率和代码读取方式，适合做数据记录、状态监测、教学实验和上层算法开发时查阅。

阅读建议：如果您只想快速拿到稳定可用的数据流，建议优先阅读“IMU 惯性测量单元”“关节编码器”和“综合示例：传感器数据记录到 CSV”。

本章介绍 ORIX 教育版四足机器狗所有传感器的数据类型、坐标约定、采集方式和编程接口。

7.1 传感器总览

传感器	硬件	数据内容	接口方法	更新频率	传输方式
IMU	HI91	角速度 + 四元数	<code>listen_imu()</code>	~50 Hz	串口 → gRPC 流
关节编码器	Robstride 内置	位置/速度/力矩	<code>listen_joint()</code>	~50 Hz	CAN → gRPC 流
电机状态	Robstride	在线/温度/电压/故障	<code>get_motor_status()</code>	按需查询	CAN → gRPC 单次
总线电压	Robstride	16 路母线电压 (V)	<code>get_voltage()</code>	按需查询	CAN → gRPC 单次
状态机	软件	FSM 状态字符串	<code>listen_state()</code>	事件驱动	gRPC 流

7.2 IMU 惯性测量单元

7.2.1 硬件与数据

ORIX 搭载 HI91 惯性测量单元，安装于机身躯干中心，通过串口 (`/dev/ttyUSB1`) 以约 50 Hz 频率上报数据。每帧包含：

字段	类型	单位	说明
<code>gyroscope.x</code>	float	rad/s	绕机体 X 轴角速度 (Roll)
<code>gyroscope.y</code>	float	rad/s	绕机体 Y 轴角速度 (Pitch)
<code>gyroscope.z</code>	float	rad/s	绕机体 Z 轴角速度 (Yaw)
<code>quaternion.w</code>	float	无量纲	四元数实部
<code>quaternion.x</code>	float	无量纲	四元数虚部 i

字段	类型	单位	说明
quaternion.y	float	无量纲	四元数虚部 j
quaternion.z	float	无量纲	四元数虚部 k
timestamp_ms	float	ms	数据时间戳（毫秒）

7.2.2 坐标系

IMU 数据使用**机体坐标系 (Body Frame)**:

IMU 坐标系与机体坐标系一致: +X 指向前进方向, +Y 指向左侧, -Y 指向右侧, +Z 垂直向上。

- X 轴: 指向机器人前方
- Y 轴: 指向机器人左侧
- Z 轴: 指向上方（右手坐标系）

7.2.3 四元数约定

ORIX SDK 使用 **Hamilton** 约定, 四元数顺序为 (w, x, y, z):

$$q = w + xi + yj + zk$$

- 表示世界坐标系到机体坐标系的旋转
- 单位四元数满足 $w^2 + x^2 + y^2 + z^2 = 1$
- 机器人水平静止时: $q = (1, 0, 0, 0)$

注意: ROS 等系统常用 (x, y, z, w) 顺序, 与本 SDK 不同。如需与 ROS 对接, 请手动调整字段顺序。

7.2.4 订阅 IMU 数据

```

1 from brainstem_sdk import OrixClient
2
3 dog = OrixClient("192.168.66.190")
4
5 # listen_imu() 返回一个迭代器, 持续产出 ImuData
6 for imu in dog.listen_imu():
7     gx, gy, gz = imu.gyroscope.x, imu.gyroscope.y, imu.gyroscope.z
8     qw, qx, qy, qz = (
9         imu.quaternion.w,
10        imu.quaternion.x,
11        imu.quaternion.y,
12        imu.quaternion.z,
```

```

13     )
14
15     print(f"角速度: ({gx:+.4f}, {gy:+.4f}, {gz:+.4f}) rad/s")
16     print(f"四元数: ({qw:.4f}, {qx:.4f}, {qy:.4f}, {qz:.4f})")
17     print(f"时间戳: {imu.timestamp_ms:.0f} ms")

```

`listen_imu()` 是一个阻塞式 gRPC 服务端流，循环会持续运行直到：

- 手动 `break` 跳出
- 按 `Ctrl+C` 中断
- 网络断开或服务端关闭连接

7.2.5 数据解读

角速度解读：

运动场景	gyro.x (Roll)	gyro.y (Pitch)	gyro.z (Yaw)
机器人静止	~0	~0	~0
直线行走	小幅波动	周期性波动	~0
原地左转	~0	~0	> 0
左侧倾斜	> 0	~0	~0

四元数转欧拉角（参考）：

```

1  import math
2
3  def quat_to_euler(q):
4      """Hamilton 四元数 (w,x,y,z) 转欧拉角 (roll, pitch, yaw), 单位 rad。"""
5
6      w, x, y, z = q.w, q.x, q.y, q.z
7      # Roll (X)
8      sinr = 2.0 * (w * x + y * z)
9      cosr = 1.0 - 2.0 * (x * x + y * y)
10     roll = math.atan2(sinr, cosr)
11     # Pitch (Y)
12     sinp = 2.0 * (w * y - z * x)
13     sinp = max(-1.0, min(1.0, sinp))
14     pitch = math.asin(sinp)
15     # Yaw (Z)
16     siny = 2.0 * (w * z + x * y)
17     cosy = 1.0 - 2.0 * (y * y + z * z)
18     yaw = math.atan2(siny, cosy)
19     return roll, pitch, yaw

```

7.3 关节编码器

7.3.1 关节布局

ORIX 共有 16 个自由度，每条腿 3 个旋转关节 + 1 个脚轮：

关节编号按四条腿分组：前右 FR (0-3)、前左 FL (4-7)、后右 RR (8-11)、后左 RL (12-15)。每条腿内部依次为 Hip、Thigh、Calf、Foot。

- **Hip:** 髋关节，控制腿的前后/内外摆动
- **Thigh:** 大腿关节，控制大腿抬起/放下
- **Calf:** 小腿关节，控制小腿伸展/收缩
- **Foot:** 脚轮关节，控制脚部轮毂转动

7.3.2 数据内容

每个关节上报以下数据：

字段	类型	单位	说明
id	int	—	关节编号 (0-15)
position	float	rad	当前角位置
velocity	float	rad/s	当前角速度
torque	float	Nm	当前力矩
status	int	—	状态码 (0 = 正常)

7.3.3 订阅关节数据

```

1 from brainstem_sdk import OrixClient
2
3 dog = OrixClient("192.168.66.190")
4
5 print("订阅关节数据 (Ctrl+C 退出)...")
6 try:
7     for joint in dog.listen_joint():
8         print(
9             f"Joint {joint.id:2d}: "
10            f"pos={joint.position:+.4f} rad  "
11            f"vel={joint.velocity:+.4f} rad/s  "
12            f"torque={joint.torque:+.4f} Nm  "
13            f"status={joint.status}"
14        )
15 except KeyboardInterrupt:
16     print("\n停止。")

```

```

17 finally:
18     dog.close()

```

注意：listen_joint() 逐关节上报，每次 yield 一个关节的数据。要获取完整的 16 关节快照，需要自行缓存最近一帧的所有关节数据。

```

1 latest = [None] * 16
2
3 for joint in dog.listen_joint():
4     latest[joint.id] = joint
5
6     # 检查是否收齐了所有 16 个关节
7     if all(j is not None for j in latest):
8         # 处理完整快照
9         for j in latest:
10             print(f" Joint {j.id}: {j.position:+.3f} rad")
11         latest = [None] * 16 # 重置，等待下一帧

```

7.3.4 关节位置安全限位

系统默认关节绝对限位为 3.14 rad。任一关节位置的绝对值超过此阈值将立即触发故障保护。该阈值可通过环境变量 HAN_DOG_JOINT_LIMIT_RAD 调整（设置更小的值可提供提前保护）。

7.4 电机状态

7.4.1 查询电机状态

get_motor_status() 返回 16 个 MotorInfo 对象，包含完整的电机健康信息：

```

1 from brainstem_sdk import OrixClient
2
3 dog = OrixClient("192.168.66.190")
4
5 motors = dog.get_motor_status()
6 for m in motors:
7     status = "在线" if m.online else "离线"
8     fault = f"故障码: {m.errors}" if m.errors else "无故障"
9     print(
10         f"Joint {m.id:2d}: {status} "
11         f"温度={m.temperature:.0f}C "
12         f"电压={m.voltage:.1f}V "
13         f"位置={m.position:+.3f}rad "

```



```

14         f"速度={m.velocity:+.3f}rad/s  "
15         f"力矩={m.torque:+.3f}Nm  "
16         f"{fault}"
17     )

```

7.4.2 MotorInfo 字段说明

字段	类型	单位	说明
id	int	–	关节编号 (0–15)
online	bool	–	电机是否在线 (CAN 通信正常)
temperature	float	C	电机内部温度
voltage	float	V	电机母线电压
position	float	rad	当前编码器位置
velocity	float	rad/s	当前转速
torque	float	Nm	当前输出力矩
errors	list[int]	–	故障码列表，空列表表示无故障

7.4.3 故障码解读

场景	表现	处理方式
online=False	电机未响应 CAN 通信	检查 CAN 线缆连接和电源
errors 非空	电机报告故障	调用 <code>clear_motor_fault()</code> 清除
temperature > 80	电机过热	降低负载或暂停运动，等待冷却
voltage < 42	母线电压过低	立即充电，系统将自动触发低压保护

7.4.4 清除电机故障

```

1 # 清除所有电机故障
2 dog.clear_motor_fault()
3
4 # 只清除特定关节（例如前右腿的 3 个关节）
5 dog.clear_motor_fault(joint_ids=[0, 1, 2])

```

7.5 电压监控

7.5.1 读取电压

`get_voltage()` 返回 16 个浮点数，对应 16 个电机的母线电压（单位 V）：

```
1 voltages = dog.get_voltage()
2 print(f"电压范围: {min(voltages):.1f}V ~ {max(voltages):.1f}V")
3
4 for i, v in enumerate(voltages):
5     warning = " [低压!]" if v < 18.0 else ""
6     print(f"  Joint {i:2d}: {v:.1f}V{warning}")
```

7.5.2 低压保护机制

阈值	行为
< 18.0 V	系统自动执行坐下动作，等待 Grounded 状态确认后禁用电机

低压保护是不可绕过的安全机制。当任一电机报告的母线电压低于 18V，控制系统将：

1. 立即发送 SitDown 指令
2. 等待机器人进入 Grounded 状态
3. 禁用所有电机

此过程通过日志输出 LOW VOLTAGE: xx.xV < 18V -- graceful sitDown。

7.5.3 电压监控最佳实践

```
1 import time
2
3 def monitor_voltage(dog, interval=5.0, warn_threshold=20.0):
4     """周期性检查电压，提前预警。"""
5     while True:
6         voltages = dog.get_voltage()
7         min_v = min(voltages)
8         if min_v < warn_threshold:
9             print(f"[警告] 电压偏低: {min_v:.1f}V (阈值 {warn_threshold}V)")
10            if min_v < 18.0:
11                print("[严重] 已触发低压保护，请立即充电！")
12                break
13            else:
14                print(f"电压正常: {min_v:.1f}V")
15            time.sleep(interval)
```

7.6 综合示例：传感器数据记录到 CSV

以下示例同时订阅 IMU 和关节数据，记录到 CSV 文件，便于后续分析：

```

1  """传感器数据记录器 -- 同时采集 IMU 和关节数据写入 CSV。"""
2
3  import csv
4  import time
5  import threading
6  from datetime import datetime
7  from brainstem_sdk import OrixClient
8
9
10 def record_sensors(host="192.168.66.190", duration=30.0):
11     """记录传感器数据到 CSV 文件。
12
13     Args:
14         host: 机器人 IP 地址。
15         duration: 记录时长（秒）。
16     """
17     dog = OrixClient(host)
18     timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
19     stop_event = threading.Event()
20
21     # ---- IMU 记录线程 ----
22     def record_imu():
23         imu_file = f"imu_{timestamp}.csv"
24         with open(imu_file, "w", newline="") as f:
25             writer = csv.writer(f)
26             writer.writerow([
27                 "timestamp_ms",
28                 "gyro_x", "gyro_y", "gyro_z",
29                 "quat_w", "quat_x", "quat_y", "quat_z",
30             ])
31             try:
32                 for imu in dog.listen_imu():
33                     if stop_event.is_set():
34                         break
35                     writer.writerow([
36                         f"{imu.timestamp_ms:.1f}",
37                         f"{imu.gyroscope.x:.6f}",
38                         f"{imu.gyroscope.y:.6f}",
39                         f"{imu.gyroscope.z:.6f}",

```

```

40         f"{imu.quaternion.w:.6f}",
41         f"{imu.quaternion.x:.6f}",
42         f"{imu.quaternion.y:.6f}",
43         f"{imu.quaternion.z:.6f}",
44     ])
45     except Exception as e:
46         print(f"IMU 记录异常: {e}")
47     print(f"IMU 数据已保存: {imu_file}")
48
49     # ---- 关节记录线程 ----
50     def record_joints():
51         joint_file = f"joint_{timestamp}.csv"
52         with open(joint_file, "w", newline="") as f:
53             writer = csv.writer(f)
54             writer.writerow([
55                 "time_s", "joint_id",
56                 "position_rad", "velocity_rad_s", "torque_nm",
57                 "status",
58             ])
59             t0 = time.monotonic()
60             try:
61                 for joint in dog.listen_joint():
62                     if stop_event.is_set():
63                         break
64                     elapsed = time.monotonic() - t0
65                     writer.writerow([
66                         f"{elapsed:.4f}",
67                         joint.id,
68                         f"{joint.position:.6f}",
69                         f"{joint.velocity:.6f}",
70                         f"{joint.torque:.6f}",
71                         joint.status,
72                     ])
73             except Exception as e:
74                 print(f"关节记录异常: {e}")
75             print(f"关节数据已保存: {joint_file}")
76
77     # ---- 启动记录 ----
78     imu_thread = threading.Thread(target=record_imu, daemon=True)
79     joint_thread = threading.Thread(target=record_joints, daemon=True)
80
81     print(f"开始记录 {duration} 秒...")
82     imu_thread.start()

```

```

83     joint_thread.start()
84
85     try:
86         time.sleep(duration)
87     except KeyboardInterrupt:
88         print("\n手动中断。")
89
90     stop_event.set()
91     dog.close()
92
93     imu_thread.join(timeout=3.0)
94     joint_thread.join(timeout=3.0)
95     print("记录完成。")
96
97
98 if __name__ == "__main__":
99     record_sensors(duration=30.0)

```

输出文件格式:

imu_20260403_143000.csv:

```

timestamp_ms,gyro_x,gyro_y,gyro_z,quat_w,quat_x,quat_y,quat_z
1234567.0,0.001234,-0.002345,0.000456,0.999800,0.012300,0.005600,0.001200
...

```

joint_20260403_143000.csv:

```

time_s,joint_id,position_rad,velocity_rad_s,torque_nm,status
0.0000,0,0.123456,0.001234,0.456789,0
0.0001,1,-0.234567,0.002345,-0.567890,0
...

```

附录：传感器数据速查表

数据项	SDK 方法	返回类型	单位	频率	备注
角速度	listen_imu()	ImuData.gyroscope	rad/s	~50 Hz	机体坐标系
姿态四元数	listen_imu()	ImuData.quaternion	无量纲	~50 Hz	Hamilton (w,x,y,z)
IMU 时间戳	listen_imu()	ImuData.timestamp_ms	ms	~50 Hz	—
关节位置	listen_joint()	JointData.position	rad	~50 Hz	绝对位置
关节速度	listen_joint()	JointData.velocity	rad/s	~50 Hz	—
关节力矩	listen_joint()	JointData.torque	Nm	~50 Hz	—

数据项	SDK 方法	返回类型	单位	频率	备注
电机在线状态	<code>get_motor_status()</code>	<code>MotorInfo.online</code>	bool	按需	CAN 通信
电机温度	<code>get_motor_status()</code>	<code>MotorInfo.temperature</code>	C	按需	内部温度
电机电压	<code>get_motor_status()</code>	<code>MotorInfo.voltage</code>	V	按需	母线电压
电机故障码	<code>get_motor_status()</code>	<code>MotorInfo.errors</code>	list[int]	按需	空 = 无故障
母线电压 (16 路)	<code>get_voltage()</code>	list[float]	V	按需	低压阈值 18V
FSM 状态变化	<code>listen_state()</code>	str	—	事件驱动	状态切换时触发

Chapter 8

API 参考

本章提供 ORIX Python SDK (brainstem_sdk) 全部公开接口的完整参考。

本章导读

本章按“查手册”的方式组织，不强调教学顺序，而强调接口签名、前置条件、返回值和常见调用场景。

阅读建议：第一次阅读时建议按“客户端构造与连接 -> 电机控制 -> 运动指令 -> 状态查询 -> 诊断与标定 -> 实时数据流”的顺序浏览，形成稳定的接口心智模型。

8.1 OrixClient 构造与连接

8.1.1 构造函数

这一节解释如何创建 SDK 客户端对象。创建成功只说明网络通道已经建立，不代表机器人已经具备执行动作的安全条件；真正发运动指令前，仍应检查状态、电压和控制权。

```
1 OrixClient(host="192.168.66.190", port=13145, timeout=5.0)
```

参数	类型	默认值	说明
host	str	"192.168.66.190"	机器人 IP 地址
port	int	13145	gRPC 服务端口
timeout	float	5.0	默认 RPC 超时时间（秒）

说明：构造函数创建一个 gRPC 不安全通道（insecure channel），连接到 host:port。通道创建即连接，无需额外调用。推荐把客户端对象视为“当前会话的控制入口”，在一个脚本或任务中尽量复用同一个连接，避免反复创建和销毁。

```
1 from brainstem_sdk import OrixClient
2
3 # 默认连接
4 dog = OrixClient("192.168.66.190")
5
6 # 自定义参数
7 dog = OrixClient("10.0.0.100", port=13145, timeout=10.0)
```

8.1.2 close()

```
dog.close() -> None
```

关闭 gRPC 通道，释放网络资源。关闭后不可再调用任何方法。

8.1.3 上下文管理器

OrixClient 支持 with 语句，退出时自动调用 close()。如果您的脚本只执行一次短任务，这通常是最稳妥的写法，因为它能避免异常退出时遗留未关闭连接。

```
1 with OrixClient("192.168.66.190") as dog:
2     dog.enable()
3     dog.stand_up()
4     dog.walk(vx=0.3)
5     dog.sit_down()
6     dog.disable()
7 # 退出 with 块后通道自动关闭
```

8.2 电机控制

8.2.1 enable()

```
dog.enable() -> None
```

使能全部 16 个电机。调用后电机通电并进入闭环控制。

项目	说明
前置条件	gRPC 连接正常，控制权未被遥控器占用
异常	grpc.RpcError – 超时或连接失败

8.2.2 disable()

```
dog.disable() -> None
```

禁用全部电机（安全停止）。电机断电，关节自由落体。

项目	说明
前置条件	无（任何状态下均可调用）
建议	在 Grounded 状态下调用，避免机器人从站立姿态跌落
异常	<code>grpc.RpcError</code> – 超时或连接失败

```

1 # 安全关机序列
2 dog.walk(vx=0.0)    # 停步
3 dog.sit_down()      # 坐下
4 # 等待机器人坐稳...
5 dog.disable()       # 禁用电机

```

8.3 运动指令

8.3.1 walk()

```
dog.walk(vx=0.0, vy=0.0, vyaw=0.0) -> None
```

发送行走速度指令。

参数	类型	默认值	范围	说明
<code>vx</code>	float	0.0	[-1.0, 1.0]	前后速度。正值 = 前进，负值 = 后退
<code>vy</code>	float	0.0	[-1.0, 1.0]	左右速度。正值 = 向左，负值 = 向右
<code>vyaw</code>	float	0.0	[-1.0, 1.0]	旋转速度。正值 = 逆时针，负值 = 顺时针

项目	说明
前置条件	机器人处于 Standing 或 Walking 状态
停步	调用 <code>dog.walk()</code> 或 <code>dog.walk(0, 0, 0)</code>
异常	<code>grpc.RpcError</code> – 超时、连接失败或 Arbiter 拒绝

速度参数为归一化值，不是物理速度。`vx=1.0` 表示当前策略允许的最大前进速度，实际物理速度取决于所加载的运动策略。

```

1 # 前进
2 dog.walk(vx=0.5)
3
4 # 向左平移
5 dog.walk(vy=0.3)
6

```

```

7 # 前进 + 左转
8 dog.walk(vx=0.3, vyaw=0.2)
9
10 # 原地停步
11 dog.walk()

```

8.3.2 stand_up()

```
dog.stand_up() -> None
```

从坐姿过渡到站立。

项目	说明
前置条件	机器人处于 Grounded 状态，电机已使能
状态变化	Grounded → Transitioning → Standing
异常	grpc.RpcError – 超时或连接失败

8.3.3 sit_down()

```
dog.sit_down() -> None
```

从站立过渡到坐姿。如果正在行走，会先停步再坐下。

项目	说明
前置条件	机器人处于 Standing 或 Walking 状态
状态变化	Standing/Walking → Transitioning → Grounded
异常	grpc.RpcError – 超时或连接失败

8.4 状态查询

8.4.1 get_state()

```
dog.get_state() -> str
```

查询当前机器人状态。

返回值	说明
"zero"	初始化 / 未使能
"grounded"	坐姿（安全状态）

返回值	说明
"standing"	站立，可接收行走指令
"walking"	行走中
"transitioning"	状态过渡中（站起 / 坐下）

```

1 state = dog.get_state()
2 print(f"当前状态: {state}")
3
4 if state == "standing":
5     dog.walk(vx=0.3)

```

8.4.2 get_voltage()

```
dog.get_voltage() -> list[float]
```

读取 16 个电机的母线电压。

项目	说明
返回值	长度为 16 的浮点数列表，单位 V
索引	对应关节编号 0-15
低压阈值	18V，低于此值系统自动触发保护

8.4.3 get_motor_status()

```
dog.get_motor_status() -> list[MotorInfo]
```

获取全部 16 个电机的健康状态。

```

1 motors = dog.get_motor_status()
2 for m in motors:
3     if not m.online:
4         print(f"Joint {m.id} 离线!")
5     if m.errors:
6         print(f"Joint {m.id} 故障: {m.errors}")
7     if m.temperature > 70:
8         print(f"Joint {m.id} 温度偏高: {m.temperature}C")

```

8.5 策略管理

8.5.1 get_profile()

```
dog.get_profile() -> ProfileInfo
```

获取当前运动策略信息及可用策略列表。

```
1 profile = dog.get_profile()
2 print(f"当前策略: {profile.current}")
3 print(f"可用策略: {profile.available}")
4 for name, desc in zip(profile.available, profile.descriptions):
5     print(f"    {name}: {desc}")
```

8.5.2 switch_profile()

```
dog.switch_profile(name: str) -> ProfileInfo
```

切换运动策略。

参数	类型	说明
name	str	目标策略名称，必须在 <code>get_profile().available</code> 列表中

项目	说明
前置条件	机器人处于 Grounded 状态
返回值	切换后的 <code>ProfileInfo</code>
异常	<code>grpc.RpcError</code> – 策略不存在、状态不对或超时

```
1 # 查看可用策略
2 profile = dog.get_profile()
3 print(f"可用: {profile.available}")
4
5 # 切换策略（必须先坐下）
6 dog.sit_down()
7 # 等待 Grounded ...
8 new_profile = dog.switch_profile("mini")
9 print(f"已切换到: {new_profile.current}")
```

8.6 诊断与标定

8.6.1 clear_motor_fault()

```
dog.clear_motor_fault(joint_ids=None) -> None
```

清除电机故障码。

参数	类型	默认值	说明
joint_ids	list[int] None	None	要清除的关节编号列表 (0-15)。None 表示清除全部

```
1 # 清除所有电机故障
2 dog.clear_motor_fault()
3
4 # 只清除前右腿
5 dog.clear_motor_fault(joint_ids=[0, 1, 2, 3])
6
7 # 只清除单个关节
8 dog.clear_motor_fault(joint_ids=[5])
```

8.6.2 set_zero()

```
dog.set_zero() -> None
```

将当前关节位置标定为零位。

项目	说明
前置条件	机器人处于 Grounded 状态
用途	出厂标定或维修后重新校准零位
注意	错误的零位会导致运动异常，非必要不调用

```
1 # 确保在 Grounded 状态下执行
2 state = dog.get_state()
3 if state == "grounded":
4     dog.set_zero()
5     print("零位标定完成。")
6 else:
7     print(f"当前状态 {state}, 请先坐下。")
```

8.7 实时数据流

所有 `listen_*` 方法返回 Python 迭代器 (`Iterator`)，底层为 gRPC 服务端流。迭代器在以下情况终止：

- 手动 `break`
- 键盘中断 (`Ctrl+C`)
- 调用 `dog.close()` 关闭通道
- 网络断开

8.7.1 `listen_imu()`

```
dog.listen_imu() -> Iterator[ImuData]
```

订阅 IMU 实时数据流，约 50Hz。

项目	说明
产出	<code>ImuData</code> (含 <code>gyroscope</code> , <code>quaternion</code> , <code>timestamp_ms</code>)
频率	~50 Hz
阻塞	是 (在 <code>for</code> 循环中阻塞等待下一帧)

8.7.2 `listen_joint()`

```
dog.listen_joint() -> Iterator[JointData]
```

订阅关节实时数据流。

项目	说明
产出	<code>JointData</code> (含 <code>id</code> , <code>position</code> , <code>velocity</code> , <code>torque</code> , <code>status</code>)
频率	~50 Hz (逐关节上报)
阻塞	是

8.7.3 `listen_state()`

```
dog.listen_state() -> Iterator[str]
```

订阅状态机变化事件。仅在状态切换时产出，不定频。

项目	说明
产出	状态字符串: <code>"zero"</code> , <code>"grounded"</code> , <code>"standing"</code> , <code>"walking"</code> , <code>"transitioning"</code>

项目	说明
频率	事件驱动（仅状态变化时触发）
阻塞	是

8.8 数据类型参考

8.8.1 Vec3

三维向量。

```
1 @dataclass
2 class Vec3:
3     x: float = 0.0
4     y: float = 0.0
5     z: float = 0.0
```

8.8.2 Quat

四元数（Hamilton 约定：w, x, y, z）。

```
1 @dataclass
2 class Quat:
3     w: float = 1.0    # 实部
4     x: float = 0.0    # 虚部 i
5     y: float = 0.0    # 虚部 j
6     z: float = 0.0    # 虚部 k
```

8.8.3 ImuData

IMU 惯性测量数据帧。

```
1 @dataclass
2 class ImuData:
3     gyroscope: Vec3    # 角速度 (rad/s)
4     quaternion: Quat   # 姿态四元数 (Hamilton w, x, y, z)
5     timestamp_ms: float # 时间戳 (ms)
```

8.8.4 JointData

单个关节的实时数据。

```

1 @dataclass
2 class JointData:
3     id: int          # 关节编号 (0-15)
4     position: float  # 角位置 (rad)
5     velocity: float  # 角速度 (rad/s)
6     torque: float    # 力矩 (Nm)
7     status: int      # 状态码 (0=正常)

```

8.8.5 MotorInfo

电机完整健康状态。

```

1 @dataclass
2 class MotorInfo:
3     id: int          # 关节编号 (0-15)
4     online: bool     # 是否在线
5     temperature: float # 温度 (C)
6     voltage: float   # 母线电压 (V)
7     position: float  # 编码器位置 (rad)
8     velocity: float  # 转速 (rad/s)
9     torque: float    # 输出力矩 (Nm)
10    errors: list[int] # 故障码列表 (空=无故障)

```

8.8.6 ProfileInfo

运动策略信息。

```

1 @dataclass
2 class ProfileInfo:
3     current: str      # 当前激活的策略名称
4     available: list[str] # 所有可用策略名称
5     descriptions: list[str] # 各策略说明文字

```

8.8.7 RobotState

状态机常量（字符串）。

```

1 class RobotState:
2     ZERO = "zero"
3     GROUNDED = "grounded"
4     STANDING = "standing"
5     WALKING = "walking"

```



```
6     TRANSITIONING = "transitioning"
```

使用方式:

```
1 from brainstem_sdk import RobotState
2
3 state = dog.get_state()
4 if state == RobotState.STANDING:
5     dog.walk(vx=0.3)
```

8.9 错误处理

OrixClient 的所有方法在遇到错误时会抛出 SDK 异常类: OrixConnectionError (网络不通)、OrixTimeoutError (超时)、InvalidStateError (状态错误)、OrixError (其他错误)。高级用户直接使用底层 stub 时才会遇到 grpc.RpcError。

8.9.1 连接超时

```
1 from brainstem_sdk import OrixClient, OrixConnectionError,
   InvalidStateError
2
3 try:
4     dog = OrixClient("192.168.66.190")
5     dog.stand_up()
6     dog.walk(vx=0.5)
7 except OrixConnectionError:
8     print("无法连接机器人")
9 except InvalidStateError as e:
10    print(f"状态错误: {e}")
```

8.9.2 常见错误码

gRPC 状态码	含义	常见原因
UNAVAILABLE	服务不可达	机器人未开机、IP 错误、服务未启动
DEADLINE_EXCEEDED	请求超时	网络延迟、服务端无响应
FAILED_PRECONDITION	前置条件不满足	状态不对 (如在 Zero 状态下调用 walk)
PERMISSION_DENIED	权限被拒	Arbiter 控制权被遥控器占用
INTERNAL	服务端内部错误	硬件异常或程序错误

8.9.3 Arbiter 控制权冲突

ORIX 的控制仲裁器（ControlArbiter）确保遥控器始终具有最高优先级。当遥控器在线时，gRPC 指令可能被拒绝：

- 遥控器在线且正在操作：gRPC 运动指令将被忽略
- 遥控器在线但空闲超过 3 秒：gRPC 自动获得控制权
- 遥控器离线：gRPC 始终拥有控制权

```
1 import grpc
2 import time
3
4 def safe_walk(dog, vx=0.0, vy=0.0, vyaw=0.0, retries=3):
5     """带重试的行走指令。"""
6     for attempt in range(retries):
7         try:
8             dog.walk(vx=vx, vy=vy, vyaw=vyaw)
9             return True
10        except grpc.RpcError as e:
11            if e.code() == grpc.StatusCode.PERMISSION_DENIED:
12                print(f"控制权被遥控器占用，等待 ({attempt+1}/{retries}) ...")
13                time.sleep(1.0)
14            else:
15                raise
16    print("无法获得控制权。请确认遥控器已松开操控杆。")
17    return False
```

8.9.4 流式接口异常处理

```
1 try:
2     for imu in dog.listen_imu():
3         # 处理数据...
4         pass
5 except grpc.RpcError as e:
6     if e.code() == grpc.StatusCode.UNAVAILABLE:
7         print("连接断开，尝试重连...")
8     else:
9         print(f"流异常: {e.code()}")
10 except KeyboardInterrupt:
11     print("用户中断。")
12 finally:
13     dog.close()
```

附录：方法速查表

分类	方法	返回值	说明
连接	<code>OrixClient(host, port, timeout)</code>	–	构造并连接
	<code>close()</code>	None	关闭连接
电机	<code>enable()</code>	None	使能全部电机
	<code>disable()</code>	None	禁用全部电机
运动	<code>walk(vx, vy, vyaw)</code>	None	行走指令
	<code>stand_up()</code>	None	坐姿 → 站立
	<code>sit_down()</code>	None	站立 → 坐姿
状态	<code>get_state()</code>	str	查询 FSM 状态
	<code>get_voltage()</code>	list[float]	16 路母线电压
	<code>get_motor_status()</code>	list[MotorInfo]	16 个电机健康状态
策略	<code>get_profile()</code>	ProfileInfo	当前策略信息
	<code>switch_profile(name)</code>	ProfileInfo	切换运动策略
诊断	<code>clear_motor_fault(joint_ids)</code>	None	清除电机故障
	<code>set_zero()</code>	None	标定零位
	<code>ping()</code>	bool	检查连接状态
速度模式	<code>set_high_speed(enabled)</code>	None	开启/关闭高速模式（最大 2.5 m/s）
	<code>get_speed_mode()</code>	str	返回“normal”或“high_speed”
手势	<code>list_gestures()</code>	list[GestureInfo]	列出所有预设动作
	<code>play_gesture(name, timeout)</code>	None	播放预设动作（默认超时 30s）
数据流	<code>listen_imu()</code>	Iterator[ImuData]	IMU 实时流
	<code>listen_joint()</code>	Iterator[JointData]	关节实时流
	<code>listen_state()</code>	Iterator[str]	状态变化流

附录：OrixCamera 速查表

OrixCamera 是可选的摄像头辅助模块（依赖 `opencv-python`），提供拍照、录像和 MJPEG 流功能。

方法	返回值	说明
<code>open()</code>	None	打开摄像头设备
<code>close()</code>	None	关闭摄像头设备
<code>read()</code>	ndarray	读取单帧图像
<code>save_photo(path)</code>	None	拍照并保存到指定路径
<code>start_recording(path)</code>	None	开始录像
<code>stop_recording()</code>	None	停止录像
<code>stream_mjpeg(host, port)</code>	None	启动 MJPEG HTTP 流服务

使用示例：

```
1 from brainstem_sdk import OrixCamera
2
3 with OrixCamera() as cam:
4     cam.save_photo("photo.jpg")
```

Chapter 9

安全指南

本章导读

本章不是附录，而是实际调机和教学演示前必须执行的操作约束。很多“偶发故障”本质上都来自跳过检查、错误顺序或在不稳定环境中直接上动作。

阅读建议：任何涉及站起、行走、调零、清故障和切换策略的操作，建议先完成本章中的检查清单，再开始编写或运行脚本。

ORIX 教育版四足机器狗是一台具有高扭矩电机的自主运动设备。操作不当可能造成设备损坏或人员伤害。请在使用前仔细阅读本章全部内容。

9.1 操作前检查清单

每次启动机器人前，必须逐项确认以下事项：

序号	检查项	确认方法	通过标准
1	电池电量充足	<code>dog.get_voltage()</code>	所有电机电压 $\geq 24V$
2	操作区域净空	目视检查	周围 1.5 米内无人员/易碎物品
3	机器人放置在地面	目视检查	四脚均接触平坦地面
4	急停装置就绪	确认遥控器在手	遥控器已开机，电量充足
5	CAN 总线连接牢固	目视检查	所有 CAN 线缆插头无松动
6	控制服务已启动	SSH 检查	<code>systemctl status brainstem</code> 显示 active
7	电机状态正常	<code>get_motor_status()</code>	16 个电机全部在线，无故障
8	关节位置合理	<code>get_motor_status()</code>	所有关节位置绝对值 $< 1.0 \text{ rad}$
9	遥控器优先级确认	操控遥控器摇杆	遥控器能正常控制机器人
10	运行环境温度适宜	环境检查	0-40°C 室温，非湿滑地面

完整的启动前检查代码：

```
1 from brainstem_sdk import OrixClient
2
3 dog = OrixClient("192.168.66.190")
4
5 # 1. 检查电压
6 voltages = dog.get_voltage()
7 min_v = min(voltages)
```

```
8 print(f"电压: {min_v:.1f}V", "-- 通过" if min_v >= 24.0 else "-- 不通过!  
   请充电")  
9  
10 # 2. 检查电机状态  
11 motors = dog.get_motor_status()  
12 offline = [m for m in motors if not m.online]  
13 faulted = [m for m in motors if m.errors]  
14  
15 if offline:  
16     print(f"离线电机: {[m.id for m in offline]} -- 不通过!")  
17 else:  
18     print("电机在线: 全部 16/16 -- 通过")  
19  
20 if faulted:  
21     print(f"故障电机: {[m.id for m in faulted]} -- 尝试清除...")  
22     dog.clear_motor_fault()  
23 else:  
24     print("电机故障: 无 -- 通过")  
25  
26 # 3. 检查关节位置  
27 bad_joints = [m for m in motors if abs(m.position) > 1.0]  
28 if bad_joints:  
29     print(f"关节位置异常: {[m.id, f'{m.position:.2f}rad'] for m in  
   bad_joints}]")  
30 else:  
31     print("关节位置: 正常 -- 通过")  
32  
33 # 4. 检查温度  
34 hot = [m for m in motors if m.temperature > 60]  
35 if hot:  
36     print(f"温度偏高: {[m.id, f'{m.temperature:.0f}C'] for m in hot}]")  
37 else:  
38     print("电机温度: 正常 -- 通过")  
39  
40 # 5. 检查状态  
41 state = dog.get_state()  
42 print(f"当前状态: {state}", "-- 通过" if state == "grounded" else "-- 注  
   意: 非 Grounded")  
43  
44 dog.close()
```

9.2 低压保护

9.2.1 保护机制

当任一电机报告的母线电压低于 **18V** 时，系统自动触发低压保护：

低压保护的完整流程如下：

1. 检测到任一电机母线电压低于 18V
2. 自动发送 SitDown 指令
3. 等待机器人进入 Grounded 状态（安全着地）
4. 禁用全部电机（Disable）
5. 日志输出：LOW VOLTAGE: xx.xV < 18V -- graceful sitDown

此保护是优雅坐下流程，不是直接断电。系统会等待机器人安全着地后才禁用电机，避免从站立姿态跌落。

9.2.2 电压等级参考

电压范围	状态	建议
$\geq 24V$	正常	正常使用
20V – 24V	偏低	关注电量，准备充电
18V – 20V	低电量	尽快结束任务
< 18V	危险	自动触发保护，必须立即充电

9.2.3 注意事项

- 低压保护不可通过 SDK 绕过
- 大电流运动（快速行走、爬坡）会导致瞬时压降，可能在电池未耗尽时触发保护
- 建议在电压低于 24V 时即结束运动任务
- 触发低压保护后，必须充电至 24V 以上才可重新启动

9.3 关节限位保护

9.3.1 保护机制

每个关节的位置都受到绝对限位保护。默认限位为 **3.14 rad**（即 π ，约 180 度）。任一关节位置绝对值超过此阈值将立即触发故障（Fault）。

配置	环境变量	默认值	说明
关节限位	HAN_DOG_JOINT_LIMIT_RAD	3.14	绝对值限位，单位 rad

9.3.2 触发后果

关节超限后：

1. 对应电机立即进入故障状态
2. 机器人可能失去该腿的控制，导致摔倒
3. 必须手动清除故障后才能恢复

9.3.3 预防措施

- 确保零位标定正确（`set_zero()` 在标准坐姿下执行）
- 不要在关节卡住或受阻的情况下强行运动
- 如果需要更严格的保护，可设置更小的限位值：

```
1 export HAN_DOG_JOINT_LIMIT_RAD=2.5 # 约 143 度
```

注：控制系统内部环境变量名仍沿用历史名称 `HAN_DOG_*`，这是为了向后兼容既有部署脚本。

9.4 电机故障处理

9.4.1 检测故障

```
1 motors = dog.get_motor_status()
2 for m in motors:
3     if m.errors:
4         print(f"Joint {m.id} 故障码: {m.errors}")
5     if not m.online:
6         print(f"Joint {m.id} 离线 (CAN 通信中断)")
```

9.4.2 清除故障

```
1 # 方法 1: 清除所有电机故障
2 dog.clear_motor_fault()
3
4 # 方法 2: 只清除特定关节
5 dog.clear_motor_fault(joint_ids=[0, 1, 2])
```

9.4.3 故障处理流程

发现电机故障后，推荐按以下步骤处理：

1. 判断机器人是否正在运动中。

2. 如果在运动中：先调用 `dog.sit_down()` 安全坐下，等待进入 *Grounded* 状态，再调用 `dog.clear_motor_fault()` 清除故障，最后重新检查电机状态。
3. 如果未在运动中：直接调用 `dog.clear_motor_fault()` 清除故障，然后重新检查。
4. 如果故障清除成功，可恢复正常运动；如果故障持续存在，需检查硬件（CAN 线缆、电源、电机本体）。

9.4.4 反复出现的故障

如果 `clear_motor_fault()` 后故障立即重新出现，说明存在持续性硬件问题：

表现	可能原因	处理
单个电机反复故障	CAN 线缆松动	重新插拔对应 CAN 接头
多个电机同时故障	母线问题	检查电池和配电板
故障码伴随高温	电机过热	停止运动，等待冷却至 50°C 以下
关节位置跳变后故障	编码器异常	重新标零 <code>set_zero()</code>

9.5 急停程序

按紧急程度从低到高，有三种急停方式：

9.5.1 软件急停（SDK）

适用于程序可控的紧急情况：

```

1 # 优雅急停：停步 -> 坐下 -> 禁用
2 dog.walk(vx=0.0, vy=0.0, vyaw=0.0)
3 dog.sit_down()
4 # 等待进入 Grounded ...
5 dog.disable()
```

9.5.2 遥控器急停

适用于现场操作人员发现危险时：

1. 立即操控遥控器摇杆 – 遥控器自动获得最高控制权，接管机器人
2. 通过遥控器操作坐下
3. 遥控器上的急停按钮可直接断开电机使能

注意：遥控器始终具有最高优先级，可在任何时刻覆盖 SDK 指令。

9.5.3 硬件断电

适用于最紧急的情况（机器人失控、硬件冒烟等）：

1. 直接关闭电池开关或拔出电池
2. 机器人将立即断电跌落
3. 注意：断电跌落可能造成机械损伤，仅在绝对必要时使用

9.5.4 急停优先级

急停方式按优先级从高到低排列：

优先级	方式	说明
1	硬件断电	物理级，不可覆盖
2	遥控器控制	Arbiter 保证最高软件优先级
3	SDK <code>disable()</code>	gRPC 调用
4	低压自动保护	系统自动触发

9.6 Arbiter 控制仲裁安全

9.6.1 仲裁规则

ORIX 的 ControlArbiter（控制仲裁器）实施以下安全规则：

规则	说明
遥控器优先	遥控器的优先级永远高于 SDK 指令，3 秒无操作自动释放
超时释放	遥控器停止操作 3 秒后，gRPC 自动获得控制权
无抢占	gRPC 不能强行抢占遥控器的控制权
超时可配	超时时间通过 <code>HAN_DOG_ARBITER_TIMEOUT</code> 配置，默认 3 秒

9.6.2 对 SDK 开发者的影响

当 SDK 发送 `walk()` 等运动指令时，Arbiter 首先检查当前控制权归属。如果 gRPC 拥有控制权，指令正常执行；如果遥控器正在占用控制权，指令将被忽略或拒绝。

- 当遥控器在线并活跃时，SDK 的运动指令将被忽略
- 状态查询（`get_state()`, `get_voltage()` 等）不受 Arbiter 影响，始终可用
- 遥控器释放后约 3 秒，SDK 自动恢复控制权，无需额外操作

9.6.3 安全启示

- 现场测试时务必携带遥控器，作为最后的安全手段
- 不要依赖 SDK 作为唯一的急停方式
- 如果 SDK 指令无响应，先检查遥控器是否在线

9.7 禁止操作

以下操作可能导致设备损坏或人员伤害，**严格禁止**：

序号	禁止操作	原因
1	在桌面、高台上运行机器人	摔落损坏，可能伤人
2	电机使能状态下翻转机器人	电机强力对抗将损坏齿轮箱
3	跳过 <i>Grounded</i> 状态直接启用电机运动	状态机保护被绕过
4	在电机使能状态下插拔 CAN 线缆	可能导致 CAN 总线短路
5	长时间运行而不监控温度	电机过热会永久损坏绕组
6	在关节卡死时反复发送运动指令	电流持续上升，烧毁电机
7	在充电状态下运行电机	部分充电器不支持边充边放
8	修改关节限位为极大值	失去关节保护
9	在湿滑/不平地面运行而无人看护	跌倒后持续运动可能损坏关节
10	多个 SDK 客户端同时发送运动指令	指令冲突导致运动抖动

9.8 标准关机序列

每次使用结束后，必须执行以下关机序列：

```
1 import time
2
3 # 步骤 1: 停止行走
4 dog.walk(vx=0.0, vy=0.0, vyaw=0.0)
5
6 # 步骤 2: 坐下
7 dog.sit_down()
8
9 # 步骤 3: 等待进入 Grounded 状态
10 for state in dog.listen_state():
11     if state == "grounded":
12         break
13
14 # 步骤 4: 禁用电机
15 dog.disable()
16
17 # 步骤 5: 关闭连接
18 dog.close()
19
20 print("安全关机完成。")
```

简化版（带超时保护）：

```
1 import time
2
3 def safe_shutdown(dog, timeout=10.0):
4     """安全关机，带超时保护。"""
5     try:
6         dog.walk()
7         dog.sit_down()
8
9         # 轮询等待 Grounded，而非使用流式接口
10        deadline = time.monotonic() + timeout
11        while time.monotonic() < deadline:
12            if dog.get_state() == "grounded":
13                break
14            time.sleep(0.2)
15
16        dog.disable()
17    except Exception as e:
18        print(f"关机异常: {e}")
19        # 最后手段：直接禁用
20        try:
21            dog.disable()
22        except Exception:
23            pass
24    finally:
25        dog.close()
```

关键原则：先坐稳再断电。直接调用 `disable()` 而不先 `sit_down()` 会导致机器人从站立姿态自由跌落。

9.9 故障诊断树

遇到问题时，按以下诊断树逐步排查：

遇到机器人异常时，按以下步骤逐级排查：

1. 机器人是否有响应？

- 无响应：检查电源（电池是否有电、开关是否打开）、检查网络连通性（ping 测试）、检查 CAN 线缆连接、SSH 是否可连通、brainstem 服务是否运行。
- 有响应：进入下一步。

2. 机器人是否在运动中？

- 在运动中：检查运动是否正常。如运动异常，参考下方运动异常诊断。
- 未运动：调用 `dog.get_state()` 查询状态。如可查询，再用 `dog.get_motor_status()` 检查电机状态和故障码；如不可查询，排查网络问题。

3. 有故障码：调用 `dog.clear_motor_fault()` 清除后重新检查。

4. 无故障码：检查状态机当前状态是否正确。

运动异常诊断：

运动异常诊断步骤：

1. 机器人摔倒：首先用 `get_voltage()` 检查电压。低于 18V 说明是低压保护触发；电压正常则检查关节限位是否超限。
2. 运动不流畅：用 `get_motor_status()` 检查电机温度。温度超过 70°C 说明过热，需暂停运动等待冷却。
3. 指令无效：检查 Arbiter 状态，确认遥控器是否在线占用控制权。

附录：安全相关环境变量

环境变量	默认值	说明
HAN_DOG_JOINT_LIMIT_RAD	3.14	关节绝对限位 (rad)
HAN_DOG_ARBITER_TIMEOUT	3	遥控器释放后 gRPC 获取控制权等待时间 (s)
HAN_DOG_SHUTDOWN_TIMEOUT	8	关机等待超时 (s)
HAN_DOG_STARTUP_TIMEOUT	10	FSM 启动等待 Grounded 超时 (s)
HAN_DOG_SENSOR_LOW_THRESHOLD	3	传感器低阈值（连续低读数计数）

附录：安全操作总结

注意：ORIX 安全操作五条铁律：

1. 永远带遥控器 — 遥控器是最终安全手段
2. 先坐再断电 — `disable()` 前必须 `sit_down()`
3. 关注电压 — 低于 24V 准备收工
4. 监控温度 — 超过 70°C 立即休息
5. 地面运行 — 永远在地面操作，永远不上桌

Chapter 10

常见问题

本章导读

本章按现场排障顺序组织，优先覆盖连接、控制权、电压、状态机和传感器这五类高频问题。

阅读建议：排障时请先确认现象，再执行对应的小脚本或检查命令，不要一开始就同时修改网络、服务和代码，否则很难定位根因。

本章汇总使用 ORIX Python SDK 过程中的常见问题及解决方案。

10.1 连接问题

处理连接问题时，建议按“网络是否可达 -> 服务是否存活 -> 端口是否开放 -> 控制脚本是否超时配置合理”的顺序逐层检查。这样可以最快把问题范围收敛到某一个环节，而不是在 Python 代码层面盲目重试。

10.1.1 Q: 连接超时 (DEADLINE_EXCEEDED)

现象：调用任何 SDK 方法时抛出 `grpc.RpcError`，状态码 `DEADLINE_EXCEEDED`。

排查步骤：

```
1 # 1. 确认网络可达
2 ping 192.168.66.190
3
4 # 2. 确认 gRPC 端口开放
5 # Linux/macOS:
6 nc -zv 192.168.66.190 13145
7 # Windows:
8 Test-NetConnection -ComputerName 192.168.66.190 -Port 13145
9
10 # 3. 确认 brainstem 服务正在运行
11 ssh sunrise@192.168.66.190 "systemctl status brainstem"
```

常见原因与解决：

原因	解决
机器人未开机	打开电池开关，等待系统启动（约 30 秒）
开发机和机器人不在同一网段	确认两台设备在同一局域网
brainstem 服务未启动	SSH 登录后运行 <code>systemctl start brainstem</code>
防火墙拦截	确认开发机防火墙放行 13145 端口
超时时间太短	构造时增大 timeout: <code>OrixClient(host, timeout=10.0)</code>

10.1.2 Q: 连接被拒绝 (UNAVAILABLE)

现象: `grpc.RpcError` 状态码 UNAVAILABLE, 提示 Connection refused。

排查:

```

1 import grpc
2
3 try:
4     dog = OrixClient("192.168.66.190")
5     dog.get_state()
6 except grpc.RpcError as e:
7     print(f"状态码: {e.code()}")
8     print(f"详情: {e.details()}")

```

常见原因:

- brainstem 服务未启动或崩溃 – 检查 `systemctl status brainstem`
- 端口被其他服务占用 – 检查 `ss -tlnp | grep 13145`
- IP 地址错误 – 确认机器人实际 IP

10.1.3 Q: IP 地址不确定

如果不确定机器人 IP:

```

1 # 方法 1: 通过路由器管理页面查看 DHCP 租约
2
3 # 方法 2: 在已知网段扫描 gRPC 端口
4 # Linux (需安装 nmap):
5 nmap -p 13145 192.168.66.0/24
6
7 # 方法 3: 如果有 USB 串口连接
8 # 登录后查看 IP:
9 ip addr show

```

默认 IP 为 192.168.66.190, 如果网络配置未更改, 直接使用即可。

10.2 电机不动

10.2.1 Q: 调用了 enable() 但电机没反应

排查清单:

```
1 # 1. 确认状态
2 state = dog.get_state()
3 print(f"状态: {state}")
4 # 应该是 "grounded" 或 "zero"
5
6 # 2. 检查电机在线状态
7 motors = dog.get_motor_status()
8 offline = [m for m in motors if not m.online]
9 if offline:
10     print(f"离线电机: {[m.id for m in offline]}")
11
12 # 3. 检查故障
13 faulted = [m for m in motors if m.errors]
14 if faulted:
15     print(f"故障电机: {[m.id, m.errors] for m in faulted}")
16     # 尝试清除
17     dog.clear_motor_fault()
18
19 # 4. 检查电压
20 voltages = dog.get_voltage()
21 print(f"电压范围: {min(voltages):.1f}V ~ {max(voltages):.1f}V")
```

常见原因:

原因	解决
未调用 enable()	先调用 dog.enable()
电机有故障未清除	dog.clear_motor_fault()
CAN 线缆松动	检查并重新插紧 CAN 接头
电池电量不足	充电至 24V 以上
遥控器占用控制权	松开遥控器摇杆, 等待 3 秒

10.2.2 Q: 只有部分电机动, 其他不动

排查:

```
1 motors = dog.get_motor_status()
2 for m in motors:
```



```

3     status = "OK" if m.online and not m.errors else "FAIL"
4     print(f"Joint {m.id:2d}: {status}   online={m.online}   errors={m.errors}")

```

常见原因:

- 对应腿的 CAN 线缆断开 – 检查该腿的 CAN 总线物理连接
- 单个电机驱动板故障 – 更换后重试
- 电机过热保护 – 查看温度值, 等待冷却

10.2.3 Q: enable() 后状态没变化

enable() 使能电机, 但不会改变 FSM 状态。使能后需要手动切换状态:

```

1 dog.enable()
2 # 此时状态仍为 "grounded", 电机通电但保持坐姿
3 dog.stand_up()
4 # 现在状态变为 "standing"

```

10.3 机器人摔倒

10.3.1 Q: 行走中突然坐下

最可能原因: 低压保护触发。

```

1 # 检查电压历史
2 voltages = dog.get_voltage()
3 print(f"当前电压: {min(voltages):.1f}V")
4 # 如果 < 18V, 说明是低压保护

```

其他可能原因:

原因	诊断方法
低电压 (< 18V)	get_voltage() 返回值低于 18
电机过热	get_motor_status() 中 temperature > 80°C
关节超限	get_motor_status() 中 position 绝对值过大
CAN 通信中断	get_motor_status() 中部分电机 online=False
传感器断开	查看 brainstem 日志中的 SEVERE 信息

查看系统日志:

```

1 # SSH 登录机器人查看日志
2 ssh sunrise@192.168.66.190

```

```

3 # 日志文件在 logs/ 目录下
4 ls logs/
5 cat logs/orix_$(date +%Y%m%d).log | tail -50

```

10.3.2 Q: 站起来后立即摔倒

排查:

1. 地面是否太滑 – 在有摩擦力的地面（橡胶垫、地毯）上测试
2. 零位是否正确 – 如果关节零位偏移，站立姿态会畸形
3. 策略是否匹配 – 确认加载的运动策略与当前机器人硬件配置一致

10.4 行走指令无效

10.4.1 Q: walk() 调用成功但机器人不动

排查顺序:

```

1 # 1. 检查状态 -- 必须是 Standing 或 Walking
2 state = dog.get_state()
3 print(f"状态: {state}") # 必须是 "standing" 或 "walking"
4
5 # 2. 如果状态不对
6 if state == "grounded":
7     print("需要先站起来: dog.stand_up()")
8 elif state == "zero":
9     print("需要先使能: dog.enable() -> dog.stand_up()")
10 elif state == "transitioning":
11     print("正在过渡, 请等待...")

```

常见原因:

原因	解决
不在 Standing 状态	先调用 <code>stand_up()</code>
遥控器占用控制权	松开遥控器摇杆, 等待 3 秒超时
速度参数为 0	检查参数: <code>dog.walk(vx=0.3)</code>
策略不支持该运动	检查当前策略是否支持横移或旋转

10.4.2 Q: 遥控器在线时 SDK 指令被忽略

这是正常行为。ControlArbiter 保证遥控器始终具有最高优先级。

解决方案:

1. 关闭遥控器
2. 或松开遥控器摇杆，等待 3 秒后 SDK 自动获得控制权

```
1 import time
2
3 # 等待控制权释放
4 print("等待遥控器释放控制权...")
5 time.sleep(4) # 等待超过 arbiter timeout (默认 3 秒)
6 dog.walk(vx=0.3) # 现在应该生效
```

10.5 策略切换失败

10.5.1 Q: switch_profile() 报错

前置条件：策略切换只能在 **Grounded** 状态下执行。

```
1 state = dog.get_state()
2 if state != "grounded":
3     print(f"当前状态 {state}，需要先坐下。")
4     dog.walk() # 停步
5     dog.sit_down() # 坐下
6     # 等待到 grounded
7     import time
8     for _ in range(50):
9         if dog.get_state() == "grounded":
10             break
11         time.sleep(0.2)
12
13 # 现在可以切换
14 profile = dog.switch_profile("mini")
15 print(f"已切换到: {profile.current}")
```

10.5.2 Q: 策略名称不存在

```
1 # 先查看可用策略
2 profile = dog.get_profile()
3 print(f"可用策略: {profile.available}")
4 print(f"策略说明: {profile.descriptions}")
5
6 # 使用列表中的名称
7 dog.switch_profile(profile.available[0])
```

10.5.3 Q: 切换策略后行为异常

每个策略针对特定硬件配置和运动模式进行训练。切换策略后如果出现异常运动：

1. 确认策略与当前硬件配置匹配
2. 立即坐下：`dog.sit_down()`
3. 切回之前正常工作的策略

10.6 IMU 数据异常

10.6.1 Q: 四元数数值看起来不对

Hamilton vs ROS 约定：

ORIX SDK 使用 Hamilton 约定：(w, x, y, z)

```
1 imu = next(iter(dog.listen_imu()))
2 # SDK 返回：(w, x, y, z)
3 print(f"w={imu.quaternion.w}, x={imu.quaternion.x}, "
4       f"y={imu.quaternion.y}, z={imu.quaternion.z}")
```

如果你的应用使用 ROS 约定 (x, y, z, w)，需要手动转换：

```
1 # SDK (Hamilton) -> ROS
2 ros_quat = (imu.quaternion.x, imu.quaternion.y,
3             imu.quaternion.z, imu.quaternion.w)
4
5 # 如果使用 scipy
6 from scipy.spatial.transform import Rotation
7 r = Rotation.from_quat([
8     imu.quaternion.x,
9     imu.quaternion.y,
10    imu.quaternion.z,
11    imu.quaternion.w,
12 ]) # scipy 使用 (x,y,z,w) 顺序
13 euler = r.as_euler('xyz', degrees=True)
```

10.6.2 Q: 机器人静止但角速度不为零

IMU 的陀螺仪存在零偏 (bias)，静止时读数不会严格为零。典型零偏范围：

典型零偏范围： $|\text{gyro.x}| < 0.02 \text{ rad/s}$ 属于正常； $|\text{gyro.x}| > 0.05 \text{ rad/s}$ 可能需要重新标定。

如需高精度角速度，可在启动时采集 100 帧静止数据取均值作为零偏，后续数据减去此偏移。

10.6.3 Q: listen_imu() 频率不是 50Hz

IMU 标称频率约 50Hz，实际可能在 48–52Hz 波动。这是正常的。如果频率显著偏低（< 30Hz），检查：

- 串口连接是否正常
- brainstem 服务是否负载过高
- 网络延迟（gRPC 传输开销）

10.7 性能与延迟

10.7.1 Q: gRPC 调用延迟有多大？

调用类型	典型延迟	说明
单次查询 (get_state() 等)	1–5 ms	局域网内
运动指令 (walk() 等)	1–5 ms	局域网内
首次连接	50–200 ms	包含 HTTP/2 握手
流式数据 (listen_imu() 等)	< 1 ms/帧	建立后持续推送

10.7.2 Q: 50Hz 控制频率够用吗？

ORIX 的运动控制循环固定为 50Hz（20ms 周期）。SDK 发送的指令在下一个控制周期生效。这意味着：

- 指令最大延迟：20ms（一个控制周期）
- 不需要以超过 50Hz 的频率发送 walk() 指令
- 建议的 walk() 指令发送频率：10–50Hz

```
1 import time
2
3 # 不需要这样做（浪费资源）：
4 while True:
5     dog.walk(vx=0.3)
6     time.sleep(0.001) # 1000Hz -- 过高，无意义
7
8 # 推荐做法：
9 while True:
10    dog.walk(vx=0.3)
11    time.sleep(0.05) # 20Hz -- 足够
```

10.7.3 Q: Python GIL 会影响实时性吗？

Python 的全局解释器锁（GIL）对 gRPC 调用影响很小，因为 gRPC 底层使用 C 扩展进行网络 I/O，不受 GIL 限制。但以下场景需要注意：

- **多线程同时调用：**gRPC 客户端本身是线程安全的，但应避免多线程同时发送运动指令（语义冲突）
- **数据处理密集：**如果在 `listen_imu()` 回调中做大量 numpy 计算，可能导致帧处理延迟。建议将数据处理放到独立线程

```
1 import threading
2 import queue
3
4 # 推荐模式：生产者-消费者
5 data_queue = queue.Queue(maxsize=100)
6
7 def imu_reader(dog):
8     """生产者：读取 IMU 数据放入队列。"""
9     for imu in dog.listen_imu():
10         try:
11             data_queue.put_nowait(imu)
12         except queue.Full:
13             data_queue.get() # 丢弃最旧的
14             data_queue.put_nowait(imu)
15
16 def data_processor():
17     """消费者：处理 IMU 数据。"""
18     while True:
19         imu = data_queue.get()
20         # 在这里做复杂计算...
21         pass
22
23 reader = threading.Thread(target=imu_reader, args=(dog,), daemon=True)
24 processor = threading.Thread(target=data_processor, daemon=True)
25 reader.start()
26 processor.start()
```

10.8 环境变量参考

以下环境变量在机器人端（brainstem 服务）生效，SDK 客户端不直接使用。了解这些变量有助于理解系统行为和排障。

环境变量	类型	默认值	有效范围	说明
HAN_DOG_PORT	int	13145	1-65535	gRPC 服务监听端口
HAN_DOG_IMU_PORT	str	/dev/ttyUSB1	设备路径	IMU 串口设备
HAN_DOG_CONTROLLER	str	auto	设备路径	遥控器设备（自动检测）
HAN_DOG_DEFAULT_PROFILE	str	(无)	策略名称	默认加载的运动策略
HAN_DOG_PROFILE_DIR	str	profiles/	目录路径	策略文件目录
HAN_DOG_ARBITER_TIMEOUT	int	3	1-60	遥控器释放后控制权切换 (s)
HAN_DOG_SENSOR_LOW_THRESHOLD	int	3	≥ 1	传感器低读数计数阈值
HAN_DOG_SHUTDOWN_TIMEOUT	int	8	1-300	关机等待超时 (s)
HAN_DOG_STARTUP_TIMEOUT	int	10	1-300	启动等待 Grounded 超时 (s)
HAN_DOG_JOINT_LIMIT_RAD	float	3.14	> 0	关节绝对限位 (rad)
HAN_DOG_LOG_DIR	str	logs	目录路径	日志文件目录
HAN_DOG_LOG	str	INFO	Logger 级别	日志级别
HAN_DOG_DEBUG_TUI	str	(无)	true/其他	启用调试 TUI
MEDULLA_STANDALONE	str	(无)	1/其他	启用独立模式

10.9 网络排障

10.9.1 基本连通性检查

```

1 # 第 1 步: Ping 机器人
2 ping 192.168.66.190
3 # 预期: 响应时间 < 5ms (局域网)
4
5 # 第 2 步: 检查 gRPC 端口
6 # Linux/macOS:
7 nc -zv 192.168.66.190 13145
8 # 预期输出: Connection to 192.168.66.190 13145 port [tcp/*] succeeded!
9
10 # Windows PowerShell:
11 Test-NetConnection -ComputerName 192.168.66.190 -Port 13145
12 # 预期: TcpTestSucceeded : True
13
14 # 第 3 步: 检查服务状态 (SSH)
15 ssh sunrise@192.168.66.190 "systemctl status brainstem"
16 # 预期: Active: active (running)

```

10.9.2 防火墙检查

```

1 # Linux 开发机

```

```
2 sudo iptables -L -n | grep 13145
3
4 # Windows 开发机
5 netsh advfirewall firewall show rule name=all | findstr 13145
6
7 # 临时放行 (Linux)
8 sudo iptables -A OUTPUT -p tcp --dport 13145 -j ACCEPT
9
10 # 临时放行 (Windows PowerShell, 管理员)
11 New-NetFirewallRule -DisplayName "ORIX gRPC" -Direction Outbound -
    Protocol TCP -RemotePort 13145 -Action Allow
```

10.9.3 网络拓扑要求

开发机 (192.168.66.x) 与 ORIX 机器人 (192.168.66.190) 必须处于同一局域网或交换机下, 不经过 NAT 和代理直连。

- SDK 使用 gRPC over HTTP/2, 需要直连 (不经过 HTTP 代理)
- 不支持跨 NAT 连接 (如果开发机在不同子网, 需要配置路由)
- Wi-Fi 连接可能有延迟抖动, 建议使用有线网络

10.9.4 远程开发 (通过 SSH 隧道)

如果开发机不在局域网内, 可通过 SSH 隧道连接:

```
1 # 在有局域网访问的跳板机上建立隧道
2 ssh -L 13145:192.168.66.190:13145 sunrise@192.168.66.190
3
4 # 然后 SDK 连接 localhost
5 dog = OrixClient("127.0.0.1", port=13145)
```

10.10 提交 Bug 报告

遇到无法自行解决的问题时, 请按以下格式提交 Bug 报告。

10.10.1 必要信息

1. SDK 版本:
`python -c "import brainstem_sdk; print(brainstem_sdk.__version__)"`
2. Python 版本:
`python --version`

3. 操作系统：
(例：Ubuntu 22.04 / Windows 11 / macOS 14)
4. 机器人固件版本：
(SSH 登录后查看 `brainstem` 版本)
5. 问题描述：
(清晰描述预期行为和实际行为)
6. 复现步骤：
(从连接开始，逐步列出操作)
7. 错误信息：
(完整的异常堆栈或错误输出)
8. 机器人日志：
(SSH 登录，提供 `logs/han_dog_YYYYMMDD.log` 中相关片段)

10.10.2 收集诊断信息的脚本

```
1  """ 诊断信息收集脚本。将输出复制到 Bug 报告中。 """
2
3  import sys
4  import platform
5  import grpc
6
7  try:
8      import brainstem_sdk
9      sdk_ver = brainstem_sdk.__version__
10 except Exception as e:
11     sdk_ver = f"导入失败: {e}"
12
13 print("=" * 50)
14 print("ORIX SDK 诊断信息")
15 print("=" * 50)
16 print(f"SDK 版本:      {sdk_ver}")
17 print(f"Python:         {sys.version}")
18 print(f"平台:           {platform.platform()}")
19 print(f"grpc 版本:      {grpc.__version__}")
20 print()
21
```

```
22 host = input("机器人 IP (默认 192.168.66.190): ").strip() or "
    192.168.66.190"
23
24 try:
25     from brainstem_sdk import OrixClient
26     dog = OrixClient(host, timeout=5.0)
27
28     # 状态
29     state = dog.get_state()
30     print(f"连接:      成功")
31     print(f"机器人状态: {state}")
32
33     # 电压
34     voltages = dog.get_voltage()
35     print(f"电压范围:    {min(voltages):.1f}V ~ {max(voltages):.1f}V")
36
37     # 电机
38     motors = dog.get_motor_status()
39     online_count = sum(1 for m in motors if m.online)
40     fault_count = sum(1 for m in motors if m.errors)
41     max_temp = max(m.temperature for m in motors)
42     print(f"电机在线:    {online_count}/16")
43     print(f"电机故障:    {fault_count}/16")
44     print(f"最高温度:    {max_temp:.0f}C")
45
46     # 故障详情
47     for m in motors:
48         if m.errors or not m.online:
49             print(f"  Joint {m.id}: online={m.online} errors={m.errors} "
50                   f"temp={m.temperature:.0f}C voltage={m.voltage:.1f}V")
51
52     # 策略
53     profile = dog.get_profile()
54     print(f"当前策略:    {profile.current}")
55     print(f"可用策略:    {profile.available}")
56
57     dog.close()
58
59 except grpc.RpcError as e:
60     print(f"连接:      失败")
61     print(f"错误码:      {e.code()}")
62     print(f"详情:      {e.details()}")
63 except Exception as e:
```

```
64     print(f"异常: {type(e).__name__}: {e}")
65
66     print()
67     print("=" * 50)
68     print("请将以上输出复制到 Bug 报告中。")
```

10.10.3 日志收集

```
1 # SSH 登录机器人
2 ssh sunrise@192.168.66.190
3
4 # 查看今天的日志
5 cat logs/han_dog_$(date +%Y%m%d).log
6
7 # 只看错误和警告
8 grep -E "WARNING|SEVERE" logs/han_dog_$(date +%Y%m%d).log
9
10 # 查看最近 100 行
11 tail -100 logs/han_dog_$(date +%Y%m%d).log
12
13 # 导出日志到本地（在开发机上执行）
14 scp sunrise@192.168.66.190:logs/han_dog_*.log ./han_dog_logs/
```

10.10.4 联系方式

- GitHub Issues: 在 brainstem 仓库提交 Issue
- 附上诊断脚本输出和相关日志

10.11 SSH 访问与系统服务

大多数排障操作都需要 SSH 登录机器人查看日志或重启服务。出厂镜像默认已启用 SSH，无需额外配置。

10.11.1 默认账户

项目	默认值
用户名	sunrise
密码	sunrise
IP 地址	192.168.66.190

项目	默认值
SSH 端口	22

出厂镜像默认开启 SSH 服务，无需手动启用。登录命令：

```
1 ssh sunrise@192.168.66.190
2 # 提示输入密码: sunrise
```

10.11.2 brainstem 服务管理

ORIX 的控制栈（brainstem）以 systemd 服务形式运行，开机自启。以下命令在 SSH 登录后执行：

```
1 # 查看服务状态
2 systemctl status brainstem
3
4 # 查看最近 100 行日志
5 journalctl -u brainstem -n 100
6
7 # 实时跟踪日志
8 journalctl -u brainstem -f
9
10 # 重启服务（通常在切换策略或修改配置后）
11 sudo systemctl restart brainstem
12
13 # 停止服务（调试时临时关闭控制栈）
14 sudo systemctl stop brainstem
15
16 # 启动服务
17 sudo systemctl start brainstem
```

10.11.3 无网络环境下的备选连接方式

如果 SSH 被校园网或企业防火墙拦截，或机器人未能获取 DHCP 地址，可以使用 RJ45 网线直连：

1. 用网线将开发机直接连接到机器人的 RJ45 口
2. 将开发机的以太网接口配置为静态 IP，例如 192.168.66.100/24
3. 使用默认地址连接：ssh sunrise@192.168.66.190

这种方式不依赖任何路由器或交换机，是调试现场最可靠的连接手段。

附录：快速排障速查表

症状	首先检查	可能原因	解决方案
连接超时	ping 机器人	网络不通	检查网线/Wi-Fi/IP
连接被拒	brainstem 服务状态	服务未启动	<code>systemctl start brainstem</code>
enable() 无效	<code>get_motor_status()</code>	电机离线/有故障	检查 CAN 线，清除故障
walk() 无效	<code>get_state()</code>	不在 Standing	先 <code>stand_up()</code>
walk() 被忽略	遥控器状态	Arbiter 被占用	松开遥控器，等 3 秒
突然坐下	<code>get_voltage()</code>	低压保护 (< 18V)	充电
突然摔倒	电机温度	过热/关节超限	冷却 / 重新标零
策略切换失败	<code>get_state()</code>	不在 Grounded	先 <code>sit_down()</code>
IMU 数据异常	四元数约定	Hamilton vs ROS	调整字段顺序
数据流中断	网络连接	gRPC 通道断开	重建 OrixClient
多客户端冲突	客户端数量	多 SDK 实例	保证单客户端